VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

REPORT
CAPSTONE PROJECT
SEMESTER 252 ACADEMIC YEAR 2025-2026

DEVELOPING AN AI AGENT TO SUPPORT ADMINISTRATION AND OPERATION OF HPC SYSTEMS BASED ON SLURM

MAJOR: COMPUTER SCIENCE

COUNCIL: COMPUTER SCIENCE - 2CC

SUPERVISOR(s): Assoc.Prof. Thoại Nam
Thin Nguyen Manh, PhD

——o0o——

STUDENT: Nguyễn Phúc Thanh Danh - 2252102

HO CHI MINH CITY, June 2026

Instructor's Signature

_____ Date: _____

Assoc.Prof. Thoại Nam (Instructor)
Faculty of Computer Science and Engineering

Declaration of Authenticity

The author, Nguyễn Phúc Thanh Danh, declares that this Capstone Project was composed and implemented entirely by the author under the guidance and supervision of Assoc. Prof. Thoại Nam and Thin Nguyen Manh, Ph.D. at the Faculty of Computer Science and Engineering, Vietnam National University — Ho Chi Minh City University of Technology.

In the process of researching and implementing this Capstone Project, the author has referenced multiple previous studies from other researchers. All referenced works have been fully and clearly cited in the References section. This Capstone Project has been published as a conference paper presented at C.

If there is any instance of plagiarism, the author is ready to accept the consequences. Ho Chi Minh City University of Technology — Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during this research.

Ho Chi Minh City, June 2026

Author,

Nguyễn Phúc Thanh Danh

Acknowledgement

First and foremost, the author would like to express his sincere gratitude to Assoc. Prof. Thoại Nam and Thin Nguyen Manh, Ph.D. for dedicating their valuable time to guide, orient, and encourage him throughout the outline and implementation phases of this Capstone Project. Their deep expertise, perceptive feedback, and constant support have been indispensable to the success of this work.

He is also thankful to the lecturers at the Faculty of Computer Science and Engineering, Vietnam National University — Ho Chi Minh City University of Technology, for providing him with essential foundational knowledge. This background has served as a strong springboard, enabling him to complete this thesis and contribute to the Vietnamese Computer Science community.

In addition, the author would like to acknowledge his own efforts during the thesis-writing process. Although taking the first step was challenging, he remained determined. By creating a practical plan and committing wholeheartedly to it, he was able to stay on track and complete the project on time.

Last but not least, he extends heartfelt thanks to his family for their unwavering support throughout his university journey. His parents' diligence has been his greatest source of motivation, encouraging him to work hard and persevere.

Abstract

High-Performance Computing (HPC) clusters have become essential infrastructure for scientific research, engineering simulations, and large-scale data processing. The Slurm Workload Manager is widely used across HPC environments and provides sophisticated resource management capabilities, but its command-line interface can create usability barriers for users who need to inspect queues, diagnose jobs, submit workloads, or perform controlled state-changing operations. This thesis addresses these challenges by designing and implementing an intelligent agent system that provides a natural language interface for Slurm cluster management.

The proposed Slurm Agent is a domain-specific control and assistance system for translating natural-language intent into concrete Slurm operations. Rather than treating an LLM as a general chatbot, the system combines typed Slurm tools, an Observer/Operator workflow with human confirmation for state-changing actions, local Slurm documentation retrieval, stateful confirmation handling, and trace-oriented evaluation. The implementation uses a React + Vite presentation layer, a session-aware FastAPI agent backend, an MCP protocol/tool layer, and real or mock Slurm infrastructure, but these frameworks serve as the substrate for Slurm-specific safety, grounding, and evaluation mechanisms.

This thesis makes six main contributions. **(1)** It defines a broad natural-language Slurm operation surface spanning monitoring, diagnosis, submission, job control, accounting, QoS/resource limits, reservations, scheduler health, configuration inspection, documentation support, and edge-case safety. **(2)** It implements a safety-centered Observer/Operator workflow that separates read-only analysis from state-changing execution through structured handoffs, human-in-the-loop confirmation, pending-action persistence, dynamic tool filtering, and session-scoped cleanup. **(3)** It develops a grounded Slurm assistance layer that combines live MCP command results for cluster state with local official-documentation retrieval for command syntax, state/reason-code interpretation, and policy-like explanations, reducing reliance on model memory alone. **(4)** It contributes a 3,135-case curated Slurm benchmark dataset with all eleven categories balanced at 285 cases each, including added coverage for fairshare, scheduler health, configuration inspection, GPU/GRES behavior, job arrays, dependencies, QOS and resource-limit diagnosis, reservations, reports, licenses, topology, federation, burst buffers, triggers, local documentation retrieval, and administrative safety workflows. **(5)** It provides an architecture-aware evaluation stack and dashboard that run live agent requests against resettable mock scenarios, collect traces for tool usage, routing, confirmation behavior, response quality, and source-to-target state transitions, and optionally apply an LLM-as-judge component. **(6)** It explores domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B-Instruct) using LoRA adapters on a 4-bit quantized base (QLoRA) with role-scoped tool exposure, distilling the commercial model's evaluation traces into a local alternative that eliminates API dependency while preserving tool-calling and safety behavior.

The implementation demonstrates the feasibility of a stateful, tool-grounded natural language interface for HPC operations. The fine-tuned open-weight model achieves an 88.3% mean

weighted score on the held-out test split, closing 29% of the gap to the commercial API baseline while eliminating external API dependency. Domain-adapted open-weight models serve as viable local alternatives for structured tool-calling tasks, reducing operational cost and preserving data sovereignty. The system provides a practical foundation for AI-assisted cluster operations where usability, traceability, and controlled execution must be balanced.

Keywords: High-Performance Computing, Slurm, Large Language Models, Model Context Protocol, Multi-Agent Systems, Human-in-the-Loop, Natural Language Interface, Parameter-Efficient Fine-Tuning

List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CLI	Command-Line Interface
CoT	Chain-of-Thought
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HITL	Human-in-the-Loop
HPC	High-Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JSON-RPC	JavaScript Object Notation Remote Procedure Call
LLM	Large Language Model
LoRA	Low-Rank Adaptation
MCP	Model Context Protocol
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
QLoRA	Quantized Low-Rank Adaptation
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
RLHF	Reinforcement Learning from Human Feedback
SDK	Software Development Kit
SQL	Structured Query Language
Slurm	Simple Linux Utility for Resource Management
SSE	Server-Sent Events
TLS	Transport Layer Security
UI	User Interface
URI	Uniform Resource Identifier

Contents

List of Tables

List of Figures

Chapter 1

Introduction

1.1 Motivation

High-Performance Computing (HPC) has become an essential infrastructure in modern scientific research, engineering simulations, and large-scale data processing. HPC clusters enable researchers to tackle computationally intensive problems that would be infeasible on conventional computing systems. From climate modeling and genomics to machine learning and financial simulations, HPC systems provide the computational power necessary to drive innovation across multiple domains.

Slurm Workload Manager, originally introduced as the Simple Linux Utility for Resource Management, has emerged as a widely deployed workload manager for HPC clusters [58, 42]. SchedMD reports that Slurm is used at government laboratories, universities, and companies worldwide, and that it manages workloads for over half of the top 10 systems in the TOP500 list [40]. Slurm provides sophisticated mechanisms for job scheduling, resource allocation, and cluster monitoring, enabling efficient utilization of computational resources across thousands of nodes. The system's flexibility and scalability have made it the preferred choice for many academic institutions and commercial organizations operating large-scale computing facilities.

Despite its power and flexibility, HPC user-support literature highlights recurring needs for training, documentation, and assistance when researchers interact with complex computing environments [39]. In Slurm-based clusters, this challenge is reflected in a broad command-line interface (CLI) comprising numerous commands, each with extensive sets of options and parameters [42]. Users must master commands such as `squeue`, `sinfo`, `sbatch`, `scontrol`, and `sacct`, along with their associated flags and output formats. This complexity is compounded by the need to write job submission scripts in Bash, which requires additional technical knowledge beyond the domain expertise of many researchers.

Recent advances in Large Language Models (LLMs) have demonstrated remarkable capabilities in natural language understanding and generation [8, 30]. These models can interpret complex user queries, reason about multi-step tasks, and generate appropriate responses or actions. The emergence of LLM-based agents—systems that combine language models with tool-calling capabilities—has opened new possibilities for creating intelligent interfaces to complex systems [54, 43].

The Model Context Protocol (MCP), developed by Anthropic, provides a standardized approach for connecting LLMs to external tools and data sources [4, 5]. MCP enables the creation of modular, reusable integrations that can expose system functionality to AI agents through explicit tool interfaces. This architectural pattern separates the concerns of tool implementation from agent logic, facilitating the development of sophisticated AI-assisted applications.

However, deploying commercial LLM APIs in HPC environments raises fundamental concerns. Cluster workloads expose sensitive metadata—user submission scripts, resource allocation patterns, job failure diagnostics, and access-control policies—that institutional data governance policies prohibit from traversing external networks. Furthermore, recurring per-query API costs scale linearly with cluster user-base size. These constraints motivate the deployment of locally-hosted open-weight models that preserve data sovereignty and eliminate ongoing costs, but at the expense of model capacity: a 14-billion parameter model cannot match the tool-selection accuracy of commercial frontier models when presented with a large number of available tools simultaneously.

This research is motivated by the opportunity to bridge the gap between Slurm's powerful capabilities and user accessibility through an intelligent conversational interface that operates entirely on local infrastructure. By leveraging LLM agents connected to Slurm through MCP, and by partitioning the tool surface across specialized sub-agents to compensate for the capacity constraints of local models, we create a system that allows users to manage cluster resources using natural language queries while maintaining full data sovereignty.

1.2 Scope and Goals

1.2.1 Project Goals

The primary objective of this research is to design, implement, and evaluate an intelligent agent system for Slurm cluster management. The project aims to make Slurm more accessible through natural-language interaction, but it also contributes the evaluation infrastructure needed to measure whether such an agent behaves correctly and safely. The following goals guide the development of this solution:

Goal 1: Natural Language Interface for Broad Slurm Operations. The system shall enable users to perform a broad set of Slurm operations through natural language queries. Users should be able to inspect jobs, partitions, nodes, accounting records, QoS and resource limits, reservations, scheduler health, submit workloads, and perform human-confirmed job-control operations without explicit knowledge of Slurm command syntax. The agent must interpret user intent accurately and translate queries into appropriate Slurm commands or documentation lookups.

Goal 2: Slurm-Specific Observer/Operator Control Flow. The system shall implement a specialized agent workflow where read-only observation is separated from state-changing job management. Documentation retrieval, data visualization, and web search fallback should remain read-only support capabilities, while handoffs to state-changing execution must carry explicit action intent, target scope, and safety context.

Goal 3: Risk-Aware Slurm Tool Integration via MCP. The system shall utilize the Model Context Protocol to provide standardized tool interfaces between the LLM agents and Slurm infrastructure. The MCP server should expose Slurm functionality as discrete, typed tools, while

the agent layer classifies tools by operational risk and routes state-changing actions through confirmation before command execution.

Goal 4: Curated Slurm Benchmark Dataset. The project shall construct a broad, scorable Slurm benchmark dataset for evaluating natural-language cluster operations. The active dataset shall cover monitoring, diagnosis, action, bulk operations, submission, safety, accounting, documentation, domain-specific Slurm workflows, multi-step reasoning, and edge cases. Each case should include expected tool behavior, routing expectations, confirmation policy, and source-to-target state information where applicable. The current active benchmark contains 3,135 curated cases, balanced across eleven categories with 285 cases each.

Goal 5: Domain-Adapted Fine-Tuning. The project shall explore whether a smaller open-weight model can be domain-adapted to replicate the tool-calling and safety behavior of the commercial baseline. The approach uses parameter-efficient fine-tuning—LoRA adapters trained on a 4-bit quantized base model (QLoRA)—on traces distilled from the evaluation framework, with role-scoped tool exposure to prevent cross-role hallucination. The resulting model should operate entirely on local infrastructure, eliminating API cost and data-privacy concerns for production HPC environments.

1.3 Challenges and Solutions

Reliability and Accuracy

Challenge: Ensuring reliability and accuracy in a natural-language Slurm agent is difficult because Slurm commands depend on exact syntax, object identifiers, flags, partitions, accounts, and current scheduler state. A generated answer may sound plausible while still selecting the wrong tool, referring to the wrong job, misunderstanding a pending reason, or confusing documentation knowledge with live cluster facts.

Solutions: The system grounds user requests in typed MCP tools instead of free-form shell generation. The agent uses a ReAct-style execution pattern [57], but the important project-specific layer is the Slurm grounding around it: live state is retrieved through Slurm tools, command and state explanations are supported by local official-documentation retrieval, and under-specified requests can trigger clarification rather than forced execution.

Safety in Cluster Operations

Challenge: Slurm is a shared operational environment. Read-only inspection is low risk, but cancelling jobs, holding jobs, changing node state, modifying reservations, or updating accounting state can interrupt users and affect shared resources. These actions cannot be handled like ordinary chatbot responses.

Solutions: The system separates read-only observation from state-changing operation through an Observer/Operator workflow. The Observer handles inspection, diagnosis, accounting/resource-limit queries, reservation checks, documentation retrieval, and web-search fallback. The Operator handles state-changing work through structured handoffs and a pending-action workflow that requires explicit user confirmation before execution. Dynamic tool filtering keeps sub-

agents within their intended responsibility boundaries, and MCP parameter validation prevents arbitrary shell-command construction.

Evaluation and Dataset Construction

Challenge: Evaluating an agentic Slurm assistant requires more than text-quality scoring. There is no ready-made public dataset covering the exact Slurm workflows and tool behavior needed; the benchmark must be manually designed as synthetic scenarios spanning read-only monitoring, diagnosis, submission, job control, accounting, documentation questions, multi-step workflows, and edge cases. Beyond benchmark construction, reliable automated evaluation additionally requires: (a) deterministic source-state resets between test cases — without them, state contamination makes pass/fail scores non-reproducible; and (b) structural traces of tool calls, routing decisions, and HITL activations — because a response can describe the correct operation while still calling the wrong tool, skipping a required handoff, or failing to trigger confirmation.

Solutions: The project manually constructs a 3,135-case scenario-grid benchmark across eleven balanced categories. Each case specifies expected tool behavior, routing, confirmation policy, and source-to-target state where applicable. Mock Slurm scenarios can be reset before each case; parallel evaluation workers use isolated session identifiers and worker-specific MCP endpoints. The evaluation harness records full tool and routing traces, so failures are diagnosable rather than reduced to an opaque aggregate score.

Diversity of Slurm Workflows

Challenge: Slurm administration spans many domains: jobs, nodes, partitions, accounting, QoS, fair-share, reservations, configuration, scheduler health, job arrays, dependencies, and selected administrative actions. A narrow prototype covering only `squeue` and `scancel` would not represent the operational breadth expected from an HPC assistant.

Solutions: The MCP tool layer exposes a broad Slurm operation surface, and the benchmark mirrors that breadth across eleven balanced categories. Live or mock MCP command results remain the source of truth for cluster-state questions, while local documentation retrieval handles reference-heavy questions, keeping live facts separate from model memory.

Domain-Adapted Fine-Tuning

Challenge: Relying on commercial API models introduces per-request cost, latency, and data-privacy concerns for production HPC environments. However, smaller open-weight models suffer from two compounding limitations: (1) they do not natively understand Slurm tool-calling conventions, and (2) their tool-selection accuracy degrades as the number of available tools grows—a phenomenon documented in recent tool-use scaling studies [36]. The Slurm agent's combined tool surface comprises 64 agent-facing tools when presented to a single monolithic agent; this leads to selection confusion, where the model calls `sacct` when `squeue` is appropriate or hallucinates non-existent commands such as `scontrol_requeue`.

Solutions: The project addresses both limitations through a dual strategy. First, the Observer/Operator architecture partitions the tool surface into two focused subsets—29 read-only tools for the Observer and 40 tools for the Operator (35 action tools plus 5 discovery reads)—

reducing each agent's selection space from 64 and recovering tool-recall accuracy (empirically +10.2 percentage points on routing-neutral tasks, Section 6.2.2). Second, the project distills successful evaluation traces from the commercial model into a structured training corpus, then applies parameter-efficient fine-tuning—LoRA adapters on a 4-bit quantized base (QLoRA)—to an open-weight model (Qwen2.5-14B-Instruct). Role-scoped tool exposure ensures each training sample includes only the tools available to its role, so the model internalizes both the safety boundary and the reduced tool surface as part of its generation behavior. The resulting model runs entirely on local GPU infrastructure with no external API dependency.

1.4 Project Structure

The report is divided into seven chapters, with a brief overview of each chapter as follows:

- **Chapter 1 - Introduction:** This chapter introduces the project topic, explains the motivation for a natural-language Slurm assistant, defines the scope and objectives, discusses the main challenges, and outlines the adopted solutions.
- **Chapter 2 - Theoretical Background:** This chapter presents the foundational knowledge required for the project, including Slurm workload management, Large Language Models, agent architectures, and the Model Context Protocol.
- **Chapter 3 - Related Tools and Works:** This chapter reviews existing HPC management interfaces, LLM agent frameworks, and MCP-based tool integrations, then positions the project relative to those works.
- **Chapter 4 - Proposed Approaches:** This chapter analyzes and designs the Slurm Agent system, including requirements, system architecture, component responsibilities, and the Observer/Operator workflow.
- **Chapter 5 - Implementation:** This chapter describes the implementation of the main system components: the MCP Slurm tool server, the agent backend, the frontend interface, and supporting evaluation infrastructure.
- **Chapter 6 - Experiments:** This chapter presents the testing approach, benchmark design, evaluation criteria, and experimental results of the Slurm Agent system.
- **Chapter 7 - Conclusion:** This chapter summarizes the project outcomes, discusses limitations, and reflects on the contributions relative to the project goals.

Chapter 2

Theoretical Background

2.1 Slurm Workload Manager

Slurm Workload Manager is an open-source cluster management and job scheduling system for Linux-based high-performance computing environments [58, 42]. It is widely deployed in research and production HPC clusters, including large systems represented in the TOP500 list [40]. This project uses Slurm as the target operational domain because it exposes a rich command surface for monitoring, diagnosis, submission, accounting, and controlled cluster administration.

2.1.1 Architecture Overview

Slurm follows a controller-and-worker architecture. The central controller maintains cluster state and scheduling decisions, while node-level daemons execute and monitor workloads.

The **slurmctld** daemon is the central controller. It tracks nodes, partitions, jobs, reservations, and resource allocations. It accepts job submissions, evaluates pending work, applies scheduling policy, and dispatches jobs to available compute resources. In production clusters, a backup controller can be configured for high availability.

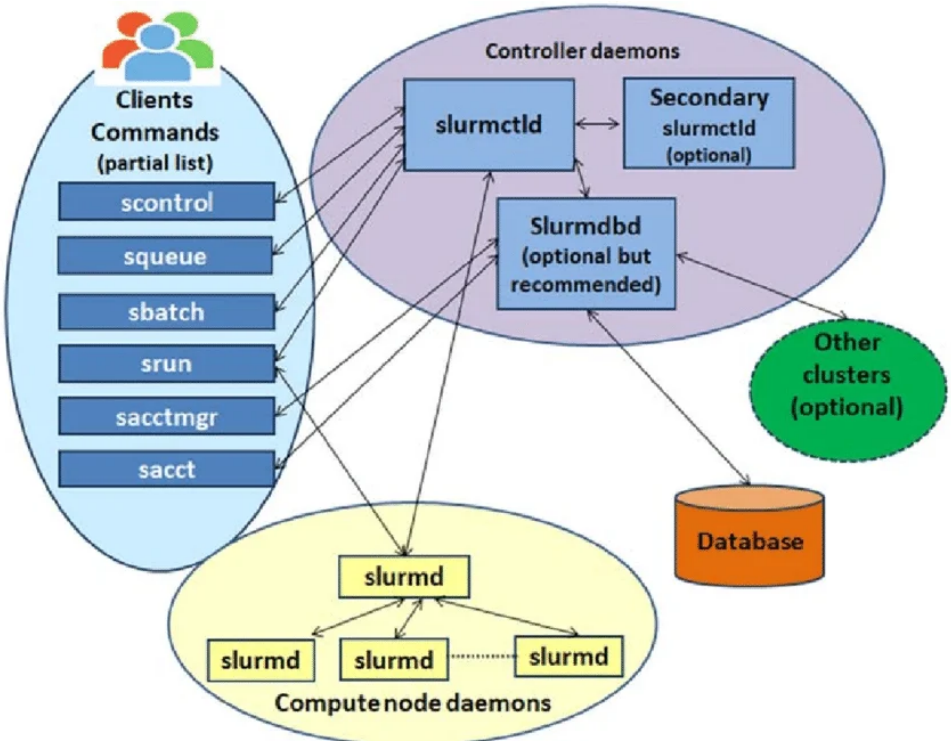


Figure 2.1: Slurm master-worker architecture. Client commands (`sbatch`, `srun`, `squeue`) submit requests to the central `slurmctld` controller daemon, which schedules work and dispatches it to per-node `slurmd` daemons on compute nodes. An optional `slurmdbd` accounting daemon records job history in a persistent database for later retrieval via `sacct`. A secondary `slurmctld` can be configured for high availability, and the controller may communicate with external clusters for multi-cluster management [1].

The **slurmd** daemon runs on each compute node. It launches job steps, monitors local execution, reports node state to the controller, and handles job control signals. This makes the compute node visible to the scheduler while preserving local execution responsibility.

The **slurmdbd** daemon provides the accounting service when enabled. It stores job history, user associations, account usage, fair-share information, and reporting data in a database. These records are important for administrative workflows, quota analysis, and historical job diagnosis.

2.1.2 Core Operational Objects

Slurm operations are built around several core objects:

Nodes are compute servers with resources such as CPUs, memory, GPUs, and generic resources. Node state matters operationally: an idle node can accept work, an allocated node is running jobs, a drained node is intentionally unavailable for new work, and a down node is unavailable because of failure or administrative action.

Partitions are logical groups of nodes. They behave like queues and encode access rules, limits, default policies, and intended usage patterns. Users often need to know which partition matches a job's resource requirements and policy constraints.

Jobs are resource allocation requests. A job includes requested resources, time limits, partition, account or QoS settings, script or command, and optional dependencies. Jobs move through states such as pending, running, completed, failed, cancelled, held, or suspended.

Steps are execution units inside an allocated job. A job may contain one or more steps, and step-level inspection is useful for understanding running work, CPU usage, memory behavior, and launched parallel tasks.

Accounts, QoS, and reservations represent policy and access constraints. They affect who can submit work, how resources are charged, which jobs receive priority, and when nodes are reserved for maintenance or special workloads.

These objects are accessed through a broad command surface. Read-only commands such as `sinfo`, `squeue`, `sacct`, and `scontrol show` inspect partitions, jobs, accounting records, nodes, reservations, and configuration state. Other commands, including `sbatch`, `srun`, `salloc`, `scancel`, selected `scontrol` operations, and `sacctmgr`, can submit work or modify jobs, nodes, reservations, accounts, and QoS policies. The same command family may contain both safe inspection operations and high-impact administrative operations; for example, `scontrol show job` is read-only, while `scontrol update` can change job or node state. This distinction is central to the Observer/Operator design used later in the thesis.

2.1.3 Why Slurm Is Difficult to Use Through Natural Language

Slurm's command-line interface is expressive, but that expressiveness creates usability and safety challenges. Users must know command names, flags, output formats, state meanings, pending reasons, account/QoS concepts, and site-specific policy conventions. Many operations are also state-dependent: cancelling a running job, releasing a held job, or updating a reservation only makes sense when the referenced object exists and is in an appropriate state.

These properties make Slurm a useful but demanding domain for a natural-language agent. The agent must map user intent to the correct command surface, obtain live cluster facts when needed, distinguish documentation questions from operational questions, and require explicit confirmation before state-changing actions are executed.

2.2 Large Language Models

Large Language Models (LLMs) are neural networks trained to model and generate natural language from large text corpora. Recent LLMs can follow instructions, summarize technical material, reason over multi-step requests, and call external tools, which makes them useful as natural-language interfaces for complex systems [8, 30]. In this thesis, the LLM is not treated as an authority on live cluster state. Instead, it is used as an intent interpretation and orchestration component that must be grounded through external tools, documentation, and structured responses.

2.2.1 Transformer-Based Language Modeling

Most modern LLMs are based on the Transformer architecture introduced by Vaswani et al. [48]. The key mechanism is self-attention, which allows each token representation to attend to other tokens in the sequence and model long-range dependencies.

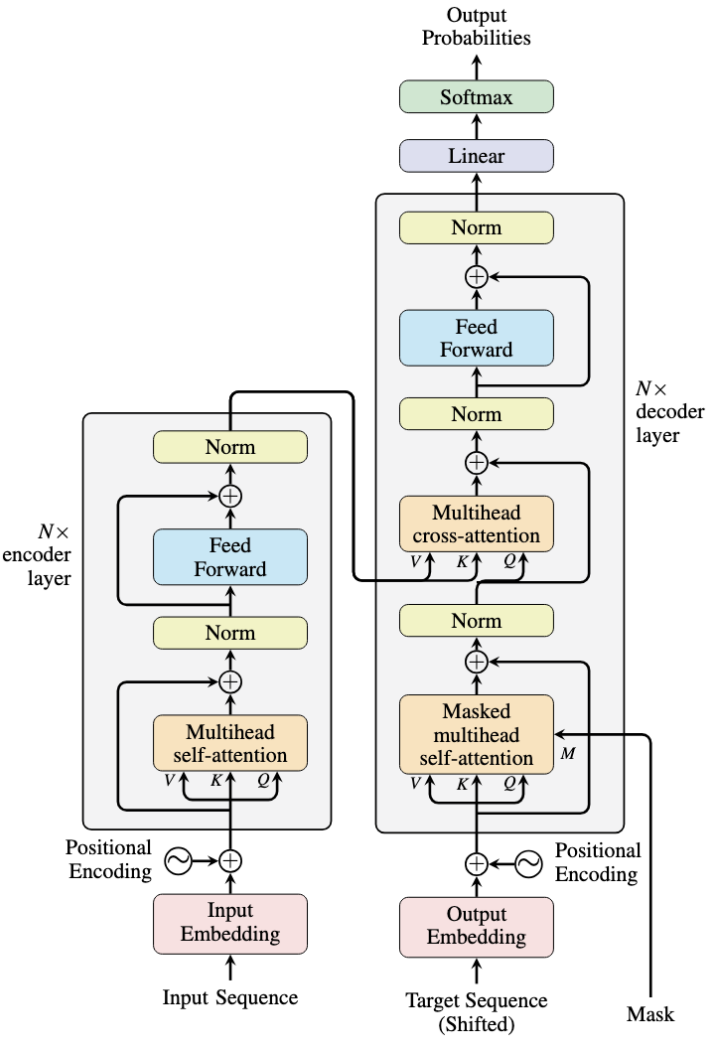


Figure 2.2: Transformer architecture as introduced by Vaswani et al. [48]. The encoder (left) and decoder (right) each consist of stacked layers combining multi-head self-attention with position-wise feed-forward networks, residual connections, and layer normalisation. Inputs are first converted to token embeddings and summed with positional encodings to inject sequence-order information. The encoder builds contextual representations of the input; the decoder attends to both its own previous outputs (masked self-attention) and the encoder output (cross-attention) to generate the target sequence token by token. Although the original design is encoder-decoder, most modern LLMs use a decoder-only variant of this structure.

In self-attention, token representations are projected into query (Q), key (K), and value (V) vectors. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the key-vector dimension. Multi-head attention performs this computation across several learned projections, allowing different heads to capture different relationships in the text. Stacked attention and feed-forward layers then produce contextual representations used for next-token prediction.

Most LLMs are trained with an autoregressive objective, where the model predicts the next token from the preceding context:

$$P(x) = \prod_{i=1}^{n} P(x_i \mid x_1, \ldots, x_{i-1}) \tag{2.2}$$

This objective allows the model to learn linguistic patterns, factual associations, programming conventions, and domain terminology from training data. However, the generated output remains probabilistic. A fluent answer can still be incomplete, stale, or unsupported by the current state of an external system.

2.2.2 Instruction Following and Alignment

Base language models are optimized for continuation, not necessarily for helpful task completion. Instruction tuning improves this behavior by training models on instruction-response examples. Reinforcement Learning from Human Feedback (RLHF) further adjusts model behavior according to human preferences for helpfulness, relevance, and safety [32].

These techniques make LLMs better suited for conversational systems, but they do not remove the need for domain constraints. In a Slurm environment, an apparently correct response may still use the wrong partition, assume outdated job state, or omit an operational precondition. Therefore, alignment must be combined with tool grounding, parameter validation, and confirmation for actions that affect cluster state.

2.2.3 Tool Use and Structured Output

Tool use extends an LLM beyond static model knowledge. Instead of answering only from its parameters, the model can choose an external function, provide structured arguments, inspect the returned result, and continue the conversation using that result. This pattern is important for operational domains because live state must be queried from the system of record.

Function calling and structured output make tool use more reliable than free-form command generation. A developer defines available tools with names, descriptions, argument schemas, and required fields. The model then emits a structured call that software can validate before execution. For example, a natural-language request such as checking a user's queued jobs can be mapped to a job-query tool rather than a raw shell string.

Structured tool calls support three important properties in this thesis:

- **Grounding**: the agent can retrieve live Slurm state instead of relying on memorized or guessed information.
- **Control**: the application can restrict the available tool set according to the current interaction mode.
- **Validation**: typed parameters can be checked before a command is executed against the cluster interface.

2.2.4 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) addresses a core limitation of LLMs: their parametric knowledge is frozen at training time and cannot reflect new documentation, version-specific syntax, or domain-specific reference material [23]. RAG augments the generative model with a retrieval step that fetches relevant passages from an external corpus before generation, grounding the output in authoritative source material.

A typical RAG pipeline consists of three stages:

1. **Indexing**: documents are split into chunks and encoded into dense vector representations using a pre-trained embedding model such as Sentence-BERT [38].
2. **Retrieval**: at query time, the user query is embedded with the same model and the top-k most similar chunks are retrieved via cosine similarity.
3. **Generation**: the retrieved chunks are prepended to the LLM prompt as context, and the model generates an answer grounded in those passages.

Lexical vs. Semantic Retrieval

Lexical retrieval (e.g., BM25, TF-IDF) matches based on token overlap between query and document. It is fast and effective for keyword-rich queries but fails when the user paraphrases concepts without using the exact terminology present in documents.

Semantic retrieval encodes both queries and documents into a shared embedding space where cosine similarity captures meaning rather than surface form. For example, the query "how to chain jobs so one runs after another" can match documentation about `--dependency=afterok` even though no keywords overlap.

Hybrid Retrieval and Reciprocal Rank Fusion

Neither method dominates across all queries. Hybrid retrieval combines both by fusing their ranked result lists. Reciprocal Rank Fusion (RRF) [10] merges rankings without requiring score normalization:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)} \tag{2.3}$$

where R is the set of retrieval systems and k is a smoothing constant (typically 60). Documents that rank highly in both lexical and semantic systems receive the highest fused score, while documents that appear in only one system are still surfaced.

This approach is well-suited for operational documentation retrieval where queries range from exact command names ("sbatch –parsable") to conceptual questions ("why is my job stuck").

2.2.5 Limitations in Operational Settings

LLMs introduce risks that are especially relevant to HPC operations. They may hallucinate nonexistent commands, merge similar options from different tools, provide outdated documentation, or infer an action that the user did not explicitly approve. They may also produce plausible explanations for failures without actually checking logs, accounting records, or scheduler state.

For these reasons, the agent architecture in this project separates language understanding from operational execution. The LLM interprets intent and decides which typed tool may be relevant, while external tools provide authoritative data and the application enforces confirmation before state-changing operations. This design uses the LLM for its strengths in language and orchestration while limiting its authority over the cluster.

2.2.6 Parameter-Efficient Fine-Tuning

Full fine-tuning of large language models updates all parameters, requiring GPU memory proportional to the full model size. For models with billions of parameters, this is impractical on commodity hardware. Parameter-Efficient Fine-Tuning (PEFT) methods address this by modifying only a small subset of parameters while keeping the base model frozen.

Low-Rank Adaptation (LoRA) [19] decomposes weight updates into low-rank matrices. For a pre-trained weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA adds a trainable decomposition:

$$W = W_0 + \Delta W = W_0 + BA \tag{2.4}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with rank $r \ll \min(d, k)$. Only A and B are trained, reducing trainable parameters from $d \times k$ to $r \times (d + k)$. A scaling factor α/r controls the magnitude of the adaptation. At inference time, the low-rank matrices can be merged back into the base weights with zero additional latency.

Quantized LoRA (QLoRA) [11] extends LoRA by quantizing the frozen base model to 4-bit precision (NormalFloat4) during training. The LoRA adapters remain in higher precision (bfloat16) for gradient computation. This reduces the memory footprint of the base model by approximately 4×, enabling fine-tuning of 14B-parameter models on a single 48 GB GPU. QLoRA introduces double quantization—quantizing the quantization constants themselves—for additional memory savings with negligible quality loss.

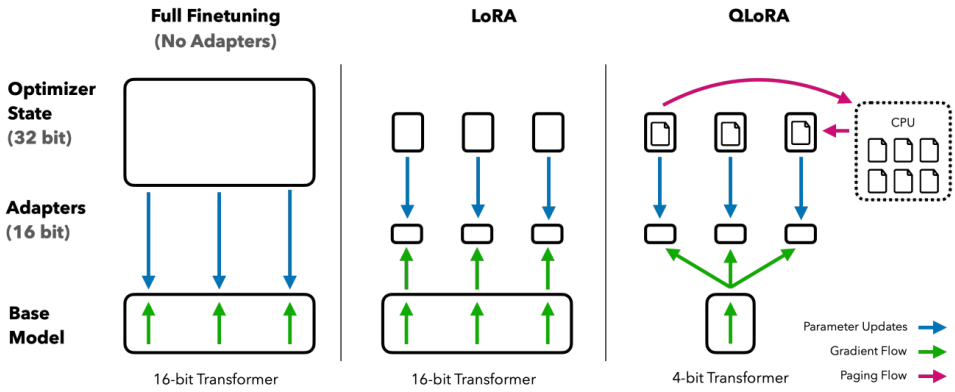


Figure 2.3: Comparison of fine-tuning approaches: (left) full fine-tuning updates all 16-bit weights with 32-bit optimizer state; (center) LoRA freezes the base model and trains low-rank adapters; (right) QLoRA quantizes the frozen model to 4-bit and uses paged optimizers to manage memory spikes [11]

Knowledge Distillation transfers capabilities from a larger teacher model to a smaller student model. In the context of tool-calling agents, distillation can use the teacher's successful execution traces as training data: the student learns to reproduce the teacher's tool selections, argument formatting, and response patterns without requiring manually annotated data [17]. This is particularly effective when the teacher model's behavior is already validated through an evaluation framework.

2.3 Model Context Protocol

The Model Context Protocol (MCP) is an open protocol for connecting AI applications to external tools, resources, and reusable prompts [4, 5]. It provides a standard boundary between a language model application and the systems it needs to inspect or operate. In this thesis, MCP is important because it allows Slurm operations to be exposed as typed tools instead of free-form shell commands.

2.3.1 Protocol Overview

MCP addresses a common integration problem in AI applications. Without a shared protocol, each application must implement its own custom connection to every database, API, command-line tool, or document source. This makes integrations difficult to reuse and makes behavior harder to inspect.

MCP uses a client-server model. A server declares the capabilities it provides, and a client connects those capabilities to an AI host application. Communication is based on JSON-RPC

2.0, which gives the protocol a structured request-response format that can be implemented across different programming languages.

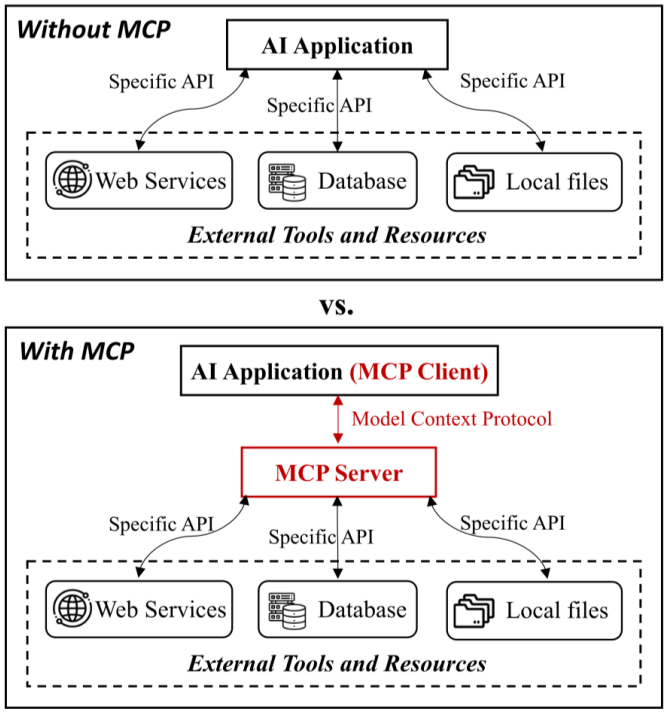


Figure 2.4: Model Context Protocol (MCP) architecture, introduced by Anthropic in late 2024 [18]. *Without MCP* (top), every AI application must implement a separate, bespoke integration for each data source or tool—a fragmented approach that is costly to build and brittle to maintain. *With MCP* (bottom), a standardised client-server protocol acts as a universal gateway: the AI application runs an **MCP Client** that communicates with one or more **MCP Servers**, each of which exposes a specific data source or capability (databases, file systems, web services, command-line tools) through a common interface. Once a server is built it is immediately compatible with any MCP-capable host, eliminating per-integration custom code and enabling a reusable, composable tool ecosystem.

The protocol does not define the internal logic of a tool. Instead, it defines how the tool is described, discovered, invoked, and returned to the host application. This makes MCP suitable for wrapping existing systems such as Slurm command-line utilities.

2.3.2 Core Components and Capabilities

MCP architecture contains three main roles:

- **MCP Hosts**: applications that users interact with directly, such as chat interfaces, IDE extensions, or desktop assistants.
- **MCP Clients**: protocol clients that maintain connections between the host and one or more servers, performing capability discovery and forwarding requests.

- **MCP Servers**: domain-specific services that expose capabilities. In this project, a Slurm MCP server exposes selected scheduler operations as named tools with defined input schemas.

Servers can expose three capability categories: **Resources** (readable data such as logs or status), **Tools** (executable operations with typed schemas), and **Prompts** (reusable workflow templates). For the Slurm Agent, tools are the primary capability—they allow Slurm operations to be represented as explicit function calls rather than free-form shell commands.

2.3.3 Tool Definition and Deployment

MCP tools are described with a name, a natural-language description, and a JSON Schema input definition. When the model selects a tool, the client sends a `tools/call` request; the server validates inputs, executes the underlying operation, and returns a structured result or error. This avoids asking the model to construct arbitrary shell commands: instead of generating a raw `squeue` string, the model calls a typed job-listing tool with validated filters. Read-only tools can be made available for observation while state-changing tools are separated into a different interaction mode requiring user confirmation. MCP supports both local deployment (server launched as a subprocess over stdio) and network deployment (HTTP transport), without changing the logical tool interface.

2.3.4 Security Considerations

MCP provides structure, but it does not automatically make tool use safe. Security must be enforced by the server and the host application. Important concerns include:

- **Access control**: only expose operations that the current user or deployment is allowed to perform.
- **Input validation**: reject missing, malformed, or out-of-policy arguments before command execution.
- **Output control**: avoid exposing sensitive user, job, or accounting information unnecessarily.
- **Auditability**: log tool invocations and results for debugging and accountability.
- **User confirmation**: require explicit approval before actions that modify cluster state.

These concerns are especially important for HPC systems because a single operation can affect shared resources, queued jobs, and other users. Therefore, this project uses MCP as the structured integration layer, while confirmation flow and mode separation provide additional control over state-changing Slurm operations.

2.4 Agent Architectures and Frameworks

Building intelligent agents requires combining Large Language Models with structured reasoning patterns and tool integration frameworks. This section examines the ReAct agent architecture and the OpenAI Agents SDK, which form the foundation of the Slurm Agent implementation.

2.4.1 ReAct: Reasoning and Acting

The ReAct paradigm, introduced by Yao et al. [57], interleaves **reasoning** (verbal chain-of-thought traces) with **acting** (tool invocations that modify or query the environment), enabling agents to solve multi-step tasks that require both planning and external interaction.

Formal Trajectory Definition

Let $\mathcal{F}$ be the set of available tools, each with a typed JSON schema Φ_f, and let $\mathcal{E}$ denote the environment (e.g., a Slurm cluster or its mock). Given a natural-language query q, an agent executes a *trajectory*:

$$\tau = \big(t_1,\, a_1,\, o_1,\;\; t_2,\, a_2,\, o_2,\;\; \ldots,\;\; t_n,\, a_n,\, o_n,\;\; r\big) \tag{2.5}$$

where:

- $t_i \in \mathcal{T}^*$ is a **thought**—a free-text reasoning trace generated by the model to decompose the task and select the next action;
- $a_i = (f_i,\, \boldsymbol{\phi}_i)$ is an **action**, consisting of a tool name $f_i \in \mathcal{F}$ and a structured argument vector $\boldsymbol{\phi}_i \in \Phi_{f_i}$ satisfying the tool's schema;
- $o_i = \mathcal{E}(a_i,\, s_i)$ is an **observation**—the response returned by the environment after executing a_i in state $s_i \in \mathcal{S}$; and
- $r \in \mathcal{T}^*$ is the **final response** emitted once the agent determines the query is resolved.

The Think-Act-Observe Loop

Each thought t_i is generated autoregressively by the agent's policy π_θ conditioned on the full interaction history up to step i:

$$t_i \;\sim\; \pi_\theta(\,\cdot \mid q,\; t_1, a_1, o_1,\; \ldots,\; t_{i-1}, a_{i-1}, o_{i-1}) \tag{2.6}$$

The action is then derived from t_i; the environment responds with o_i; and the context is extended by (t_i, a_i, o_i) for the next step. The loop terminates at step n when the model emits a text response without a further tool call, or when a maximum depth N_{max} is reached.

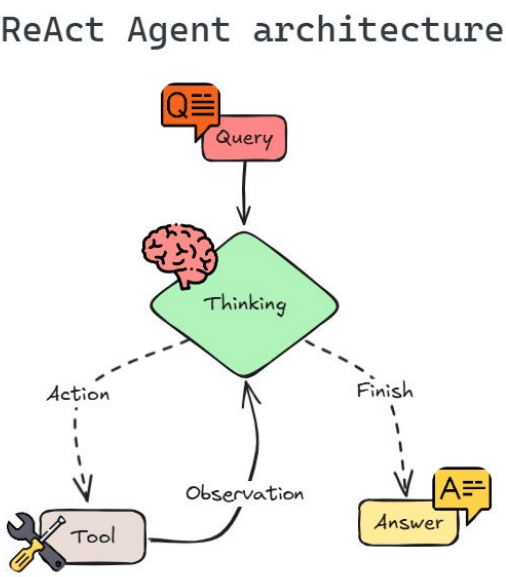


Figure 2.5: The ReAct (Reason + Act) reasoning loop [57]. Starting from a user *Query*, the agent enters an iterative cycle: it *Thinks* (produces a chain-of-thought reasoning trace to determine what information is missing), *Acts* (selects and invokes an external Tool), and *Observes* (reads the result returned by the environment). The loop exits when the agent emits a final *Answer*. Grounding reasoning in real tool observations at each step suppresses hallucination by preventing the model from fabricating facts that a tool call would contradict. [3]

ReAct is well suited for HPC operations because: (1) Slurm diagnosis requires chaining multiple queries (queue state, priority, partition availability) before a conclusion can be drawn; (2) the visible reasoning trace t_i makes each tool-selection decision auditable; and (3) observations o_i can revise the agent's plan when initial assumptions are contradicted by live cluster state.

2.4.2 OpenAI Agents SDK

The OpenAI Agents SDK [31] provides the execution framework used in this project. Its key abstractions are:

- **Agent**: An LLM with instructions, tools, and behavioral config. Separate agents isolate read-only observation from state-changing operations.
- **Runner**: Manages the execution loop—sending messages, processing tool calls, collecting results.
- **Tools**: Functions agents invoke to interact with external systems, with automatic parameter validation.
- **Handoffs**: Transfer control between specialized agents, passing conversation context.

The Runner's internal loop naturally implements ReAct: (1) collect messages, (2) send to LLM, (3) if tool calls, execute and loop; (4) if final text, return; (5) if handoff, transfer to target agent. The SDK also integrates with MCP servers, treating MCP tools as native agent tools.

Chapter 3

Related Tools and Works

This chapter situates the project within four research threads: HPC management interfaces, tool-augmented LLM agents, multi-agent safety, and agent evaluation benchmarks. No existing system combines all six properties analysed in Section 3.5; the gap motivates this work.

3.1 HPC Management Interfaces

Slurm provides expressive CLI tools (`squeue`, `sacct`, `sinfo`, `sbatch`, `scancel`) but assumes deep scheduler expertise [58, 42]. Portals such as Open OnDemand [20] and JupyterHub [22] lower barriers through graphical forms but still require manual parameter selection and provide no natural-language entry point. Monitoring stacks (Open XDMoD [33], Prometheus/Grafana [35, 15]) are observational and cannot translate requests into validated cluster operations.

Recent work integrates LLMs with HPC infrastructure. Chat AI [12] deploys LLMs *on* Slurm-backed GPUs as a service but does not use the LLM to *manage* the cluster: there is no tool-calling, no safety boundary, and no evaluation of scheduler operations. LangChain-Parsl [2] connects an LLM to HPC workflows via the Parsl parallel programming framework; it demonstrates that LLMs can drive parallel task graphs but lacks HITL confirmation, native Slurm tool integration, and domain-adapted fine-tuning. Fujitsu's AI Computing Broker [14] optimises GPU allocation at the resource-brokering layer without a natural-language interface or tool-calling evaluation. A concurrent MCP-based system for quantum-HPC hybrid execution [45] uses the same protocol as this thesis to dispatch quantum circuits to HPC backends, demonstrating MCP's applicability beyond Slurm, though it is scoped to circuit submission rather than general cluster management.

3.2 Tool-Augmented LLM Agents

The field of tool-calling LLMs has matured rapidly. Gorilla [34] fine-tuned LLaMA on APIBench to generate accurate ML framework API calls, demonstrating that domain-specific fine-tuning substantially reduces argument hallucination. ToolLLM [36] scaled this to 16,000 RESTful APIs by using ChatGPT to generate training trajectories—the same teacher-distillation paradigm used in this thesis—and showed that a domain-adapted open model can match GPT-3.5 on tool selection. Xu et al. [55] further showed that open-source LLMs require usage examples and in-context demonstration to achieve reliable tool manipulation, and that training data coverage directly determines success rate. AgentTuning [60] fine-tuned LLaMA on diverse agent trajectories to produce a generalist backbone; like ToolLLM it does not enforce safety boundaries or target a specific operational domain. More recently, GOAT [28] (ACL 2026) proposes a training

framework for goal-oriented tool use, and Trajectory2Task [51] synthesises verifiable multi-step trajectories from complex user intents—both confirm that trace-based distillation is an effective strategy for teaching multi-step tool calling to smaller models.

Across all these systems, a consistent finding emerges: the gap between base open-weight models and commercial APIs can largely be closed through domain-specific fine-tuning on execution traces. However, none of them address a safety-critical operational domain where incorrect tool calls have irreversible real-world consequences, and none include a human confirmation gate for destructive operations.

3.3 Multi-Agent Safety and Coordination

General multi-agent frameworks such as AutoGen [53] and MetaGPT demonstrate that decomposing tasks across specialised agents improves accuracy on complex planning. The Observer/Operator split in this thesis is motivated by the same principle but adds a dimension absent from general frameworks: *structural safety enforcement*. Greenblatt et al. [16] (ICML 2024) formalise AI control protocols that remain safe even under adversarial model behaviour, identifying trusted-editing and untrusted-monitoring as robust patterns. The Observer/Operator handoff instantiates this principle: the Observer classifies intent and gathers evidence (read-only, structurally constrained), while the Operator acts only on a structured payload and only after explicit human confirmation. Jamshidi et al. [21] identify tool poisoning and prompt injection as key risks in MCP deployments; this thesis mitigates both through parameter validation in the MCP layer, tool-set filtering per agent role, and the HITL gate that surfaces the exact planned action to the user before execution.

3.4 Agent Evaluation Benchmarks

AgentBench [24] (ICLR 2024) evaluates LLMs across eight environments including OS tasks and web browsing, finding a 20–40 percentage point gap between top commercial models and open-source alternatives on agentic tasks—consistent with the 25.1 pp gap observed between base Qwen and GPT-5-mini in this thesis. τ-bench [56] evaluates tool-agent-user interaction with policy compliance in retail and airline domains; it finds that even GPT-4o succeeds on fewer than 50% of tasks, confirming that domain-specific tool-plus-policy tasks are hard even for frontier models. API-BLEND [7] (ACL 2024) curates large corpora for training and testing API-task LLMs covering tool detection, slot filling, and API sequencing, reinforcing that carefully curated domain-specific datasets are both necessary and sufficient for strong tool-calling performance. ReAct [57] and Toolformer [43] establish the foundational reasoning and tool-use patterns on which all subsequent agent systems build.

No existing benchmark tests Slurm-specific tool calling, multi-agent routing, or HITL compliance. Direct numerical comparison between this thesis and prior benchmarks is structurally impossible because the task space, tool definitions, safety dimensions, and scoring methodology

are all different. The internal three-way comparison in Chapter 6.2—GPT-5-mini vs. fine-tuned Qwen vs. base Qwen—plus a monolithic architecture ablation—constitutes the controlled experimental evidence. Table 3.1 summarises the key methodological differences.

Table 3.1: Comparison of agent evaluation benchmarks and methodology.

Benchmark	Domain	Cases	Scored Dimensions	Scoring	HITL	Local Model
AgentBench [24]	General (8 envs)	∼1,750	Task success	Binary pass/fail	×	×
τ-bench [56]	Retail / Airline	∼180	Policy compliance	Binary pass/fail	∼	×
API-BLEND [7]	General API	∼10,000	Slot fill, detection	Accuracy per dimension	×	×
Ours	Slurm HPC	3,135	Tool recall, routing, HITL, state	score$(c) \geq 0.80$	✓	✓

3.5 Positioning and Gap Analysis

Table 3.2 maps related systems across six dimensions that are jointly necessary for a production Slurm assistant.

Table 3.2: Comparison of related systems. ✓ = fully supported; ∼ = partial; × = absent.

System	NL Interface	Typed Tools	HITL Safety	Domain FT	Local Deploy	Domain Benchmark
Open OnDemand [20]	×	×	N/A	N/A	✓	×
Chat AI [12]	×	×	×	×	✓	×
LangChain-Parsl [2]	✓	∼	×	×	✓	×
Fujitsu ACB [14]	×	×	N/A	×	×	×
Gorilla / ToolLLM [34, 36]	✓	✓	×	✓	✓	General
AgentTuning [60]	✓	✓	×	✓	✓	General
τ-bench [56]	✓	✓	∼	×	×	Domain
Slurm Agent (ours)	✓	✓	✓	✓	✓	✓

The table reveals a clear gap: HPC-specific systems lack tool-calling and fine-tuning; general tool-calling systems lack safety enforcement and domain evaluation; the one domain-benchmarked system (τ-bench) provides no domain-adapted local model and no confirmation gate for destructive operations. This thesis is the first to combine Slurm-native typed tool integration, structural read/write safety separation with human confirmation, domain-specific QLoRA fine-tuning, and a purpose-built 3,135-case evaluation benchmark, all deployable on a single local GPU.

Chapter 4

Proposed Approaches

4.1 Proposed Slurm Agent Overview

The proposed system is a stateful Slurm Agent for translating natural-language operational requests into grounded scheduler interactions. Its purpose is to study how an LLM-based agent can interpret Slurm intent, choose typed tools, separate read-only inspection from state-changing operations, and produce traces that can be evaluated against expected behavior. A key contribution is the domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B-Instruct) using QLoRA, enabling local deployment without reliance on commercial APIs while maintaining tool-calling accuracy through training data aligned exactly with the inference-time prompts and schemas. The user-facing interface is only an entry point; the main design object is the agent control flow, the fine-tuning pipeline, and the evaluation method around it.

The system sits between a natural-language request and the Slurm command surface. A request such as "why is my job pending?", "show failed jobs for this account", or "cancel job 12345" is not executed as a free-form shell command. Instead, the agent maps the request onto a constrained set of MCP tools, retrieves current or simulated cluster state, and decides whether the task is observational, diagnostic, documentation-oriented, or state-changing.

This objective creates four design requirements. First, the agent must ground claims in Slurm evidence rather than model memory alone. Second, commands that modify cluster state must pass through an explicit confirmation path. Third, the runtime must expose enough trace information to evaluate tool choice, routing, confirmation behavior, and state-transition consistency. Fourth, the system should operate with a locally fine-tuned open-weight model so that HPC sites retain data sovereignty and avoid per-query API costs, while still matching the behavioral quality of commercial models. The evaluation benchmark validates both the architecture itself (via the commercial baseline) and the fine-tuned model's ability to replace that baseline for local deployment.

The following sections detail the architecture that satisfies these requirements: the dual-agent routing design (Section 4.2), the formal requirements it satisfies (Section 4.3), the MCP tool protocol (Section 4.4), the ReAct reasoning cycle and fine-tuning pipeline (Section 4.5), and the scenario-grid evaluation method (Section 6.1).

4.2 System Architecture

This section presents the overall architecture of the Slurm Agent system, describing the major components, their interactions, and the rationale behind architectural decisions. The design uses common web and agent frameworks, but the architectural contribution is the Slurm-specific

control plane built on top of them: broad Slurm tool coverage, read/write separation with confirmation, and local documentation grounding.

4.2.1 Architecture Overview

The Slurm Agent is organized as a four-layer stack. At the top, a thin **Presentation Layer** accepts natural-language requests from users and relays confirmation decisions without embedding any domain logic. Below it, the **Agent Layer** houses the Observer/Operator multi-agent backend that performs ReAct reasoning and enforces safety routing—the Observer handles read-only operations and decides when to hand off to the Operator for state-changing operations. The **Protocol Layer** (MCP) partitions the full tool surface into two access classes—write-access Slurm commands gated through the Operator, and read-only monitoring commands available to the Observer—alongside documentation lookup and web search tools. At the bottom, the **Infrastructure Layer** provides the execution targets: the LLM service, the Slurm cluster, the internet for external queries, and a SQLite context store for session persistence.

This separation ensures that the boundary between reading cluster state and mutating it is an explicit architectural seam rather than an implicit convention. Read-only inspection flows entirely within the Observer path, while any state-changing operation must cross into the Operator path and pass through a confirmation gate before reaching the MCP tool layer.

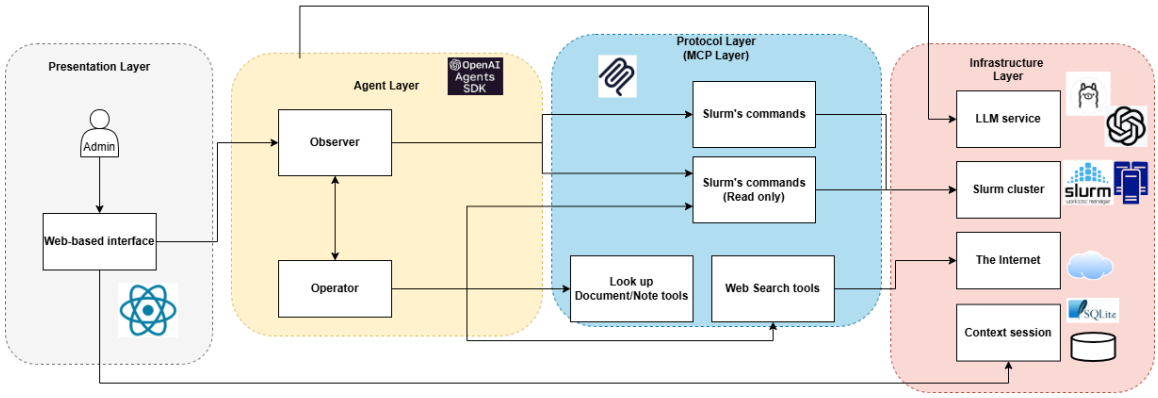


Figure 4.1: Slurm Agent System Architecture. The system is organised into four layers. (1) **Presentation Layer**: a React-based web interface through which the user submits natural-language requests. (2) **Agent Layer**: the reasoning core built on the OpenAI Agents SDK, comprising an Observer that interprets user intent and gathers context, an Operator that decides on actions and coordinates tool use, and a knowledge-base lookup tool for cluster documentation and notes. (3) **Protocol Layer**: an MCP (Model Context Protocol) server that translates agent decisions into typed, validated calls to Slurm commands (`squeue`, `scancel`, etc.) and web-search tools, decoupling agent logic from execution details. (4) **Infrastructure Layer**: the execution targets, including the LLM service (local fine-tuned model or cloud API), the Slurm cluster, the internet for external lookups, and a SQLite context store that persists session state across turns.

4.2.2 Design Rationale

The main design decision is to treat Slurm interaction as a controlled operation surface rather than as open-ended command generation. A general chatbot can explain commands, but a cluster assistant must also decide when to inspect live state, when to consult documentation, when to ask for clarification, and when to refuse or delay execution until a user confirms intent. The architecture therefore combines two forms of grounding:

- **Operational grounding**: current cluster facts come from Slurm tools exposed through MCP.
- **Documentation grounding**: command syntax, reason-code meanings, and policy-like explanations come from local Slurm documentation retrieval.

The architecture is also designed to support incomplete or ambiguous user requests. When the user says a phrase such as "cancel that job", the system must determine whether a specific job is already known from context. If the target is unclear, the safe behavior is to ask for clarification. If the target is clear but the action changes cluster state, the safe behavior is to create a pending action and require confirmation. This is why session state and pending-action persistence are architectural components rather than UI conveniences.

4.2.3 Component Descriptions

Presentation Layer

The Presentation Layer is intentionally thin. It provides a React-based web interface through which users submit natural-language requests and issue explicit confirm/cancel decisions for pending actions. This layer is responsible for:

- forwarding natural-language requests to the agent backend,
- presenting final responses and selected status events,
- relaying confirm/cancel decisions for pending actions,
- exposing traces needed for debugging and evaluation.

The presentation layer is therefore support infrastructure, not the central contribution. Agent behavior is defined entirely by the backend control flow and can be exercised either through the web interface or directly via the automated evaluator.

Agent Layer

The Agent Layer implements core reasoning and orchestration using a dual-agent architecture. Both agents share the same fine-tuned LLM and MCP tool connection but operate under distinct system prompts and tool-access policies:

- **Observer**: The primary entry point for every request. It implements the ReAct reasoning loop (Think → Act → Observe), classifies user intent, and handles all read-only operations by invoking MCP tools—queue inspection (`squeue`), node status (`sinfo`), accounting queries (`sacct`), scheduler diagnostics (`sdiag`), documentation lookup (`lookup_slurm_docs`), and web search (`web_search`, `fetch_web_content`). All of these are typed MCP tools served by the Protocol Layer; the Observer selects and calls them but does not implement them. When the request requires cluster state modification, the Observer produces a structured handoff directive specifying the action, target scope, and tool, then delegates to the Operator.

- **Operator**: Receives handoff directives for state-changing operations such as job submission, cancellation, hold, release, requeue, and configuration updates. It may perform a single discovery read to resolve targets (e.g., listing jobs before cancelling), then invokes the action tool. Execution is gated by a human-in-the-loop confirmation step for all destructive operations.

This division is central because incorrect execution can waste compute time, interrupt other users, or change shared cluster policy. The Observer never holds action tools, and the Operator never runs without an explicit handoff—ensuring that the boundary between reading and mutating state is enforced by the framework rather than by prompt compliance alone.

The handoff from Observer to Operator is a **structured LLM output**: the Observer reasons freely in natural language over the user request and the live tool catalog, then produces a typed payload with four fields that encode its decision. `action_request` is a natural-language description of what the user wants done, written by the Observer and preserved verbatim for the Operator's reasoning context. `required_tool` is the name of the specific MCP action tool that must be called (e.g., `scancel`, `sbatch`); the Observer infers this by matching the request semantics to the available tool catalog. `target_scope` is one of two values chosen by the Observer: `explicit` if the user named specific job IDs or targets, or `discovery` if the targets must be resolved at runtime by querying the cluster first (e.g., "cancel all my jobs" requires listing the user's queue before cancelling). `targets` is a list of job IDs or user identifiers the Observer has already extracted from the request or from prior tool calls; for `discovery` scope this list may be empty and is populated by the Operator's pre-action read. The entire payload is generated by the LLM in a single tool call—there is no rule-based extraction or regex parsing.

The return path gives the Observer full execution context. After the Operator executes, a handoff filter (`_observer_handoff_input_filter`) constructs a synthetic user message containing: (1) the original user request, recovered from the session history so the Observer knows what was originally asked; and (2) all deduplicated tool-output snippets from the Operator turn (each up to 2,000 characters, with no count cap). The Observer's conversation history and tools are stripped entirely—it receives only this synthetic message—together with an explicit instruction not to re-trigger the handoff. The Observer therefore sees exactly what the user asked and what was executed, enabling a complete and accurate final response. Figure 4.2 illustrates the

routing logic and HITL confirmation flow for each agent.

Beyond safety enforcement, the dual-agent design provides a measurable **tool-scope reduction** benefit for capacity-constrained local models. When all tools are combined into a single monolithic agent, the context exposes 64 agent-facing tools; the model must discriminate among semantically similar read and action tools simultaneously (e.g., `sacct` vs. `squeue` vs. `sacctmgr_list`). By partitioning tools into two focused subsets—29 read-only tools for the Observer and 40 tools for the Operator (35 action tools plus 5 discovery reads)—each agent operates within a narrower selection space where correct tool identification is easier. Ablation experiments in Section 6.2.2 quantify this effect.

Protocol Layer

The Protocol Layer implements the Model Context Protocol (MCP) server, which exposes the full Slurm command surface as 64 typed tools organised into two access classes:

- **Write-access tools** (routed through the Operator) — state-changing Slurm commands such as `sbatch`, `scancel`, `scontrol hold/release`, `scontrol update`, and accounting management. These always require an explicit HITL confirmation before the subprocess is spawned.
- **Read-only monitoring tools** (available to the Observer) — non-mutating inspection commands such as `squeue`, `sinfo`, `sacct`, `sdiag`, `sprio`, and `sstat`. These execute immediately without confirmation.
- **Documentation and web tools** — `lookup_slurm_docs` (keyword search over 273 indexed Slurm documentation pages), `web_search`, and `fetch_web_content` for external lookups. Available to the Observer only.

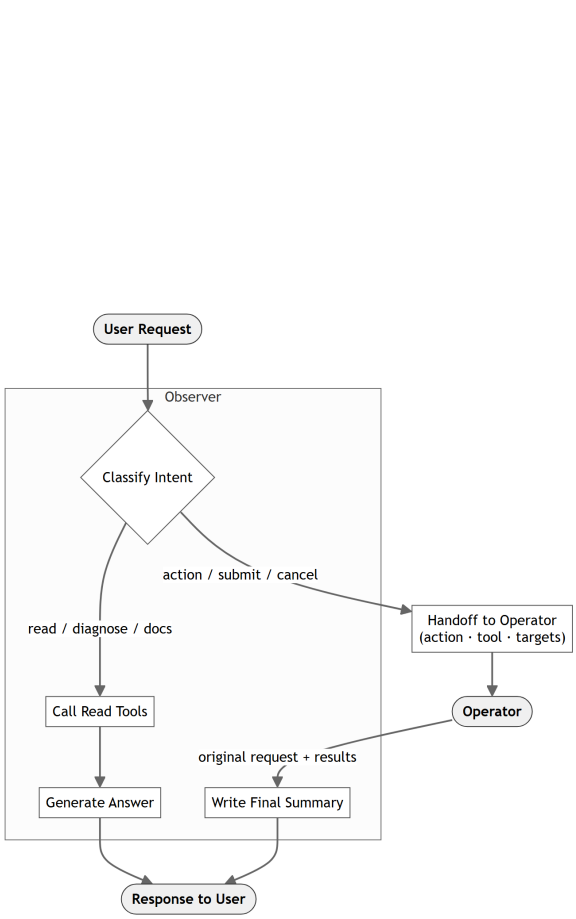
Each tool exposes a JSON schema that the agent uses to construct arguments; the layer validates and rejects malformed calls before any subprocess is spawned. The full tool surface is described in Section 4.4.

The tool-scope reduction from partitioning is shown in Figure 4.3: the Observer's 29 read-only tools and the Operator's 40 tools are each a focused subset of the 64-tool monolithic surface.

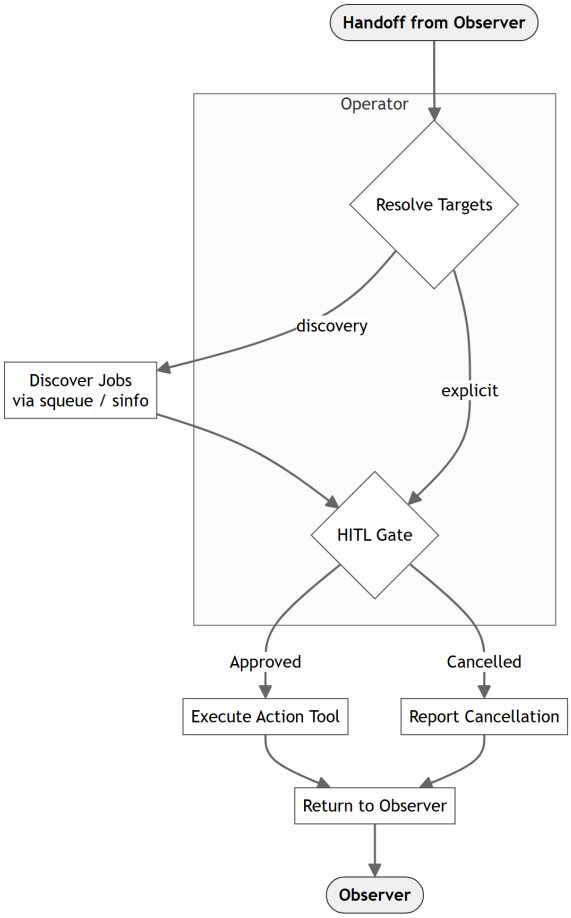
Infrastructure Layer

The Infrastructure Layer provides the four concrete execution targets that the Protocol Layer dispatches to:

- **LLM service** — local Ollama instance, self-hosted fine-tuned model behind an OpenAI-compatible endpoint, or cloud API (GPT-5-mini). All three are interchangeable through a single provider abstraction; behavioral differences reflect model capability only.
- **Slurm cluster** — real CLI access via `subprocess` calls to Slurm binaries, or a mock scenario engine for deterministic evaluation. The agent code is identical in both modes.

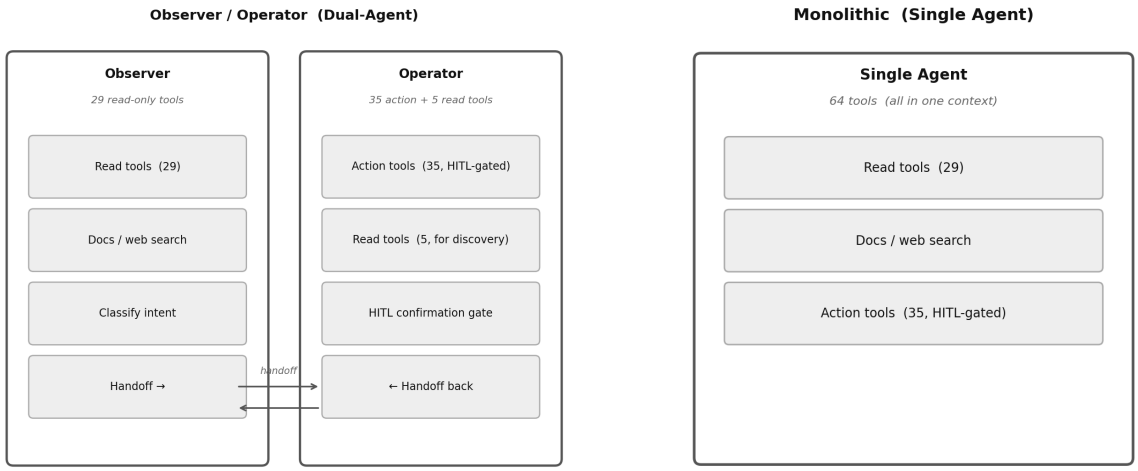


(a) Observer routing flow



(b) Operator HITL confirmation flow

Figure 4.2: **(a) Observer:** Every request enters the Observer agent, which classifies the user's intent and routes accordingly. Read-only requests (diagnose, docs, domain) are handled directly by calling the relevant MCP read tools and generating an answer. Action requests (submit, cancel, hold) trigger a structured handoff to the Operator carrying four fields: `action_request` (natural-language description), `required_tool` (MCP tool name), `target_scope` (`explicit` or `discovery`), and `targets` (list of job IDs or user identifiers). After the Operator returns, its tool output feeds back into the Observer, which generates the final response to the user. **(b) Operator:** On receiving the handoff, the Operator first resolves targets. If `target_scope` is `discovery`, it queries `squeue` or `sinfo` to enumerate the matching jobs before proceeding; if `explicit`, it uses the provided list directly. The resolved targets are presented to the user at the HITL gate—a mandatory confirmation step before any destructive tool is called. If approved, the action tool (`scancel`, `sbatch`, `scontrol`, etc.) executes and the result is returned to the Observer; if cancelled, a cancellation notice is returned instead.



(a) Observer/Operator split (b) Monolithic baseline

Figure 4.3: Tool partitioning: Observer/Operator split (left) versus Monolithic (right). The Observer receives 29 read-only MCP tools plus the handoff mechanism; the Operator receives 35 action tools and 5 discovery reads (40 total). The Monolithic baseline exposes all 64 tools in one context. Partitioning directly improves tool-call accuracy for the 14B-parameter model; ablation results are in Section 6.2.2.

- **The Internet** — external web access used by the `web_search` and `fetch_web_content` tools when local documentation is insufficient (e.g., site-specific error codes, obscure NCCL or InfiniBand failures).
- **Context session (SQLite)** — persists session state, pending actions, and conversation history across turns, enabling multi-turn interactions where a later message can reference a job mentioned earlier.

4.3 Requirements Overview

This section formalises the functional and non-functional requirements that the architecture described in Section 4.2 is designed to satisfy. Requirements are derived from the project goals in Section 4.1 and the operational needs of HPC administrators and researchers.

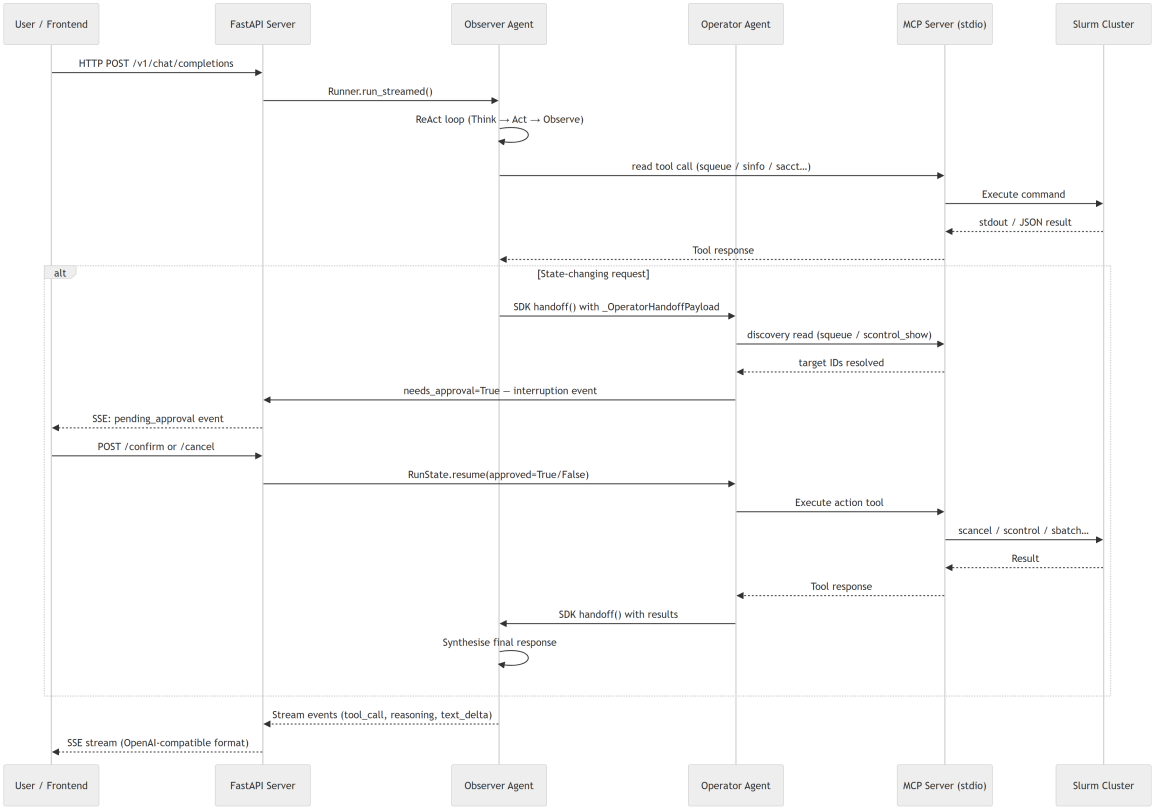


Figure 4.4: Request lifecycle data flow (sequence diagram). A user request arrives at the FastAPI server over HTTP or WebSocket. The server invokes the ReAct Agent, which classifies intent via the Router and delegates to the appropriate Observer or Operator sub-agent. The sub-agent communicates with the MCP Server over stdio, which in turn executes the relevant Slurm command against the cluster. Results propagate back up the call chain; the agent synthesises a response with embedded trace events that are streamed to the client as a Server-Sent Events (SSE) stream. Dangerous operations trigger a confirmation interruption between the sub-agent and the MCP Server before execution proceeds.

4.3.1 Functional Requirements

Table 4.1: Functional Requirements

ID	Requirement
FR1	The system shall interpret natural-language Slurm queries, including variations in phrasing and implicit context, and map them to appropriate typed tool calls.
FR2	The system shall retrieve live cluster state (queues, nodes, partitions, accounting, reservations, scheduler diagnostics) via read-only Slurm tools and present results grounded in tool output.
FR3	The system shall execute state-changing operations (submit, cancel, hold, release, requeue, node control, account management) only after explicit user confirmation.
FR4	The system shall retrieve relevant Slurm documentation from a local corpus using hybrid retrieval (lexical + semantic), with web search as a fallback when the local corpus does not cover the topic.
FR5	The system shall maintain multi-turn conversation context, enabling follow-up questions and contextual references within a session.
FR6	The system shall provide real-time feedback during execution through structured status events, making tool selections and intermediate results visible to the user.
FR7	The system shall support deterministic evaluation against mock cluster states, producing behavioral traces that can be compared to expected tool selections, routing decisions, and state transitions.

4.3.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

ID	Requirement
NFR1	**Safety**: The system shall never execute cluster-mutating commands without explicit confirmation. The confirmation mechanism shall be enforced at the tool layer, independent of LLM behavior.
NFR2	**Accuracy**: Responses shall be grounded in tool output. The system shall not fabricate cluster state, job IDs, node names, or resource figures.
NFR3	**Latency**: Simple read-only queries shall complete within interactive response times. Long-running workflows shall emit incremental status events.
NFR4	**Extensibility**: New Slurm tools shall be addable via the MCP server without modifying the agent core. Tool schemas are discovered at runtime.
NFR5	**Context Efficiency**: The system shall avoid unnecessary growth of LLM context by lazy-loading documentation, capping retrieval output size, and separating large artifacts from conversational memory.
NFR6	**Reproducibility**: Evaluation runs shall be deterministic given the same mock state, model, and test case. Provider and model shall be configurable per session.

4.4 Module Specifications

This section provides a detailed breakdown of the MCP tool surface and the evaluation scoring methodology. Observer and Operator workflow details are described in Section 4.2.

4.4.1 MCP Tool Layer

The MCP server provides 64 typed tools organized into functional groups. Table 4.3 shows the distribution across operational categories. The grouping reflects Slurm's own command families: read-only monitoring tools dominate (suitable for the Observer), while job control, node management, and accounting tools require Operator-level confirmation before execution.

Table 4.3: MCP Tool Groups

Group	Count	Examples
Queue/Monitoring	13	`squeue`, `sinfo`, `sacct`, `sdiag`, `sprio`, `sstat`
Cluster Config/Status	7	`scontrol_show_config`, `scontrol_ping`, topology, federation
Job Control	8	`sbatch`, `scancel`, `scontrol_hold`, `srun`
Node Management	6	`scontrol_node`, power up/down, features, GRES
Reservations	4	create/delete/update/show reservation
Accounting	9	`sacctmgr` list/add/modify/delete/archive/dump
Fairshare/Reports	2	`sshare`, `sreport`
Cluster Admin	4	shutdown, reconfigure, debug level, tokens
Triggers	3	`strigger_set`, `strigger_get`, `strigger_clear`
Documentation & Web	3	`web_search`, `fetch_web_content`, `read_file`
Job/Entity Update	5	`scontrol_update`, requeue, suspend, resume, reconfigure
Total	**64**	

Dual Execution Mode: The server supports *real mode* (translates tool calls to Slurm CLI commands) and *mock mode* (operates on mutable in-memory state with five predefined scenarios). Mock mode enables deterministic evaluation from known starting states.

Parameter Validation: Before execution, the MCP layer validates job IDs, node names, partition names, and scans batch scripts for dangerous patterns, providing a safety layer independent of LLM reasoning quality.

4.4.2 Evaluation Pipeline

Each benchmark test case $c \in \mathcal{C}$ specifies a tuple $(q_c, s_c^{\text{src}}, s_c^{\text{tgt}}, \mathcal{T}_c^*, h_c, \kappa_c)$ where q_c is the natural-language request, s_c^{src} and s_c^{tgt} are the source and target cluster states, $\mathcal{T}_c^*$ is the expected tool set, $h_c \in \{0, 1\}$ is the routing label (Observer-only vs. Observer→Operator handoff), and κ_c is the HITL requirement flag. The agent's response is scored across five weighted dimensions:

$$\text{score}(c) = w_1 \cdot \text{TR}(c) + w_2 \cdot \text{R}(c) + w_3 \cdot \text{H}(c) + w_4 \cdot \text{S}(c) \tag{4.1}$$

where the weights $(w_1, w_2, w_3, w_4) = (0.35,\ 0.25,\ 0.25,\ 0.15)$ sum to 1, and each component is defined as:

- $\text{TR}(c) = |\hat{\mathcal{T}}_c \cap \mathcal{T}_c^*| \,/\, |\mathcal{T}_c^*|$ — **tool recall**: fraction of ground-truth tools invoked by the agent;
- $\text{R}(c) = \mathbf{1}[\hat{h}_c = h_c]$ — **routing match**: indicator that the agent's control-flow path matches the ground-truth label;
- $\text{H}(c) = \mathbf{1}[\hat{\kappa}_c = \kappa_c]$ — **HITL match**: indicator that the confirmation gate fired (or was correctly suppressed);

- $\mathrm{S}(c) \in [0,1]$ — **state match**: fraction of expected job-state transitions correctly executed, verified against the mock cluster's post-run snapshot.

For the architecture ablation (Section 6.2.2), the monolithic baseline cannot satisfy $\mathrm{R}(c) = 1$ on any of the 237 handoff cases by construction. To enable a like-for-like quality comparison, routing is excluded and the remaining weights are re-normalised by $1/(1-0.25)$, yielding the ablation score:

$$\mathrm{score}_{\mathrm{abl}}(c) = \frac{w_1}{0.75} \cdot \mathrm{TR}(c) + \frac{w_3}{0.75} \cdot \mathrm{H}(c) + \frac{w_4}{0.75} \cdot \mathrm{S}(c) = \frac{7}{15} \cdot \mathrm{TR}(c) + \frac{1}{3} \cdot \mathrm{H}(c) + \frac{1}{5} \cdot \mathrm{S}(c) \quad (4.2)$$

Both the dual-agent and monolithic baselines are scored with $\mathrm{score}_{\mathrm{abl}}$ for the ablation comparison, so the 6.1 pp gap reflects only tool-recall and state-correctness differences. The primary evaluation metric is the mean weighted score over the test set:

$$\overline{\mathrm{score}} = \frac{1}{|\mathcal{C}_{\mathrm{test}}|} \sum_{c \in \mathcal{C}_{\mathrm{test}}} \mathrm{score}(c) \quad (4.3)$$

All reported results are averaged over $k = 3$ independent trials with temperature $= 0$, making each trial fully deterministic; the reported figure is the mean across all trials. An optional LLM-as-judge provides supplementary quality scores on a 1–5 scale reported alongside the structural metrics.

4.5 Large Language Model Architecture

The agent backbone follows the ReAct paradigm (Section 2.4): at each step, the model generates a reasoning trace t_i, invokes a tool $a_i = (f_i, \boldsymbol{\phi}_i)$, and incorporates the environment response o_i before proceeding. The two subsections below describe how prompt design and fine-tuning exploit this loop for HPC operations.

4.5.1 Instruction Design

The **Observer** prompt defines tool categories, routing rules for Operator handoff, evidence requirements (responses must cite tool output), documentation retrieval triggers, and output structure.

The **Operator** prompt defines confirmation requirements for destructive operations, scope preservation rules, discovery-read permissions, error handling, and return protocol.

Both agents share the same underlying model and MCP connection but receive strictly different tool lists at runtime—the Observer sees only the 29 read-only MCP tools plus documentation retrieval and the handoff mechanism, while the Operator sees 35 action tools plus 5 guarded discovery reads. This partitioning serves a dual purpose: it enforces safety boundaries by making it structurally impossible for the Observer to invoke destructive tools, and it reduces each agent's tool count compared to the monolithic baseline (64 tools in one context), directly addressing the

tool-selection degradation observed in 14B-parameter models when presented with large, mixed tool surfaces. The system supports multiple LLM backends (local Ollama, OpenAI API, or a self-hosted fine-tuned model behind an OpenAI-compatible endpoint) with identical prompts across all providers, so behavioral differences reflect model capability rather than configuration drift.

4.5.2 Domain-Adapted Fine-Tuning

While the system achieves strong results with commercial API models, reliance on external providers introduces latency, cost, and privacy concerns for production HPC environments. To address this, the project performs domain-adapted fine-tuning of an open-weight model (Qwen2.5-14B-Instruct) as the local agent backbone, using knowledge distilled from a capable teacher model (GPT-5-mini) through a controlled MCP mock environment.

MCP Mock Environment for Data Collection

Training an agent requires diverse, realistic tool interactions—but running against a live Slurm cluster is impractical for data generation at scale. The solution is a *mock MCP server* that simulates the full tool surface with deterministic, resettable cluster states.

The mock implements every MCP tool (queue queries, node status, accounting, job submission, cancellation, etc.) backed by in-memory state that can be configured per scenario. Each evaluation benchmark case defines an initial cluster snapshot—jobs in various states, node configurations, queue policies—and the expected agent behavior against that snapshot. This allows thousands of agent interactions to be collected without access to real hardware.

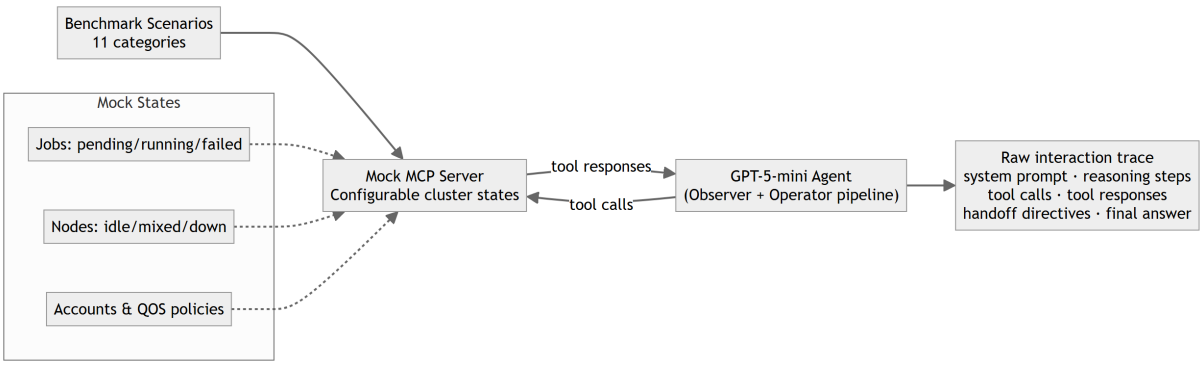


Figure 4.5: MCP mock environment for training data collection. Benchmark scenarios across 11 categories are fed to a GPT-5-mini agent running via the same Observer/Operator pipeline used at inference time. The agent interacts with a configurable mock MCP server whose in-memory cluster state covers jobs (pending, running, failed), nodes (idle, mixed, down), and account/QOS policies. The full exchange—system prompt, reasoning steps, tool calls, tool responses, handoff directives, and final answers—is captured as a raw interaction trace, which forms the distillation corpus for fine-tuning.

Distillation from GPT-5-mini

The training data is generated by running GPT-5-mini as the agent backbone against the mock MCP server across the full benchmark suite. Each successful run produces a complete interaction trace: the system prompt, user request, the model's reasoning steps, tool calls with arguments, tool responses, handoff directives, and final answers.

This distillation approach offers two advantages over manual annotation. First, the teacher model demonstrates *correct behavioral patterns*—proper tool selection, evidence-grounded responses, appropriate handoff decisions—that would be expensive to write by hand for hundreds of scenarios. Second, the traces are guaranteed to be *format-consistent* with the inference-time protocol because they are produced by the same runtime pipeline that the student model will later serve.

The pipeline processes each passing trace into the student model's chat template format: a system prompt, alternating user/assistant/tool turns, with tool calls encoded as structured JSON matching the exact schema the model must generate at inference time.

Data Processing

Successful distillation traces are processed through three steps before training:

1. **Handoff splitting.** Traces involving an Observer-to-Operator handoff are split at the handoff boundary into two independent training samples. The Observer sample ends with the handoff directive; the Operator sample begins from the handoff context. This increases Operator coverage proportionally to the frequency of multi-agent flows.

2. **Role-scoped tool filtering.** Each sample carries only the tools available to its active role—Observer samples include read-only tools plus the handoff mechanism; Operator samples include only action tools and discovery reads. This teaches the model that tool availability is conditional on role, preventing cross-role hallucination (e.g., an Observer invoking `scontrol_hold`).

3. **Deduplication.** Samples sharing identical message sequences (distinct test IDs that produced the same interaction pattern) are removed, yielding 2,625 unique training samples. The benchmark's 11 categories (read, diagnose, action, bulk, safety, submission, multi_step, account, edge, docs, domain) with multiple scenarios per category ensure the retained samples cover the full operational surface.

The resulting dataset is assembled through the following pipeline:

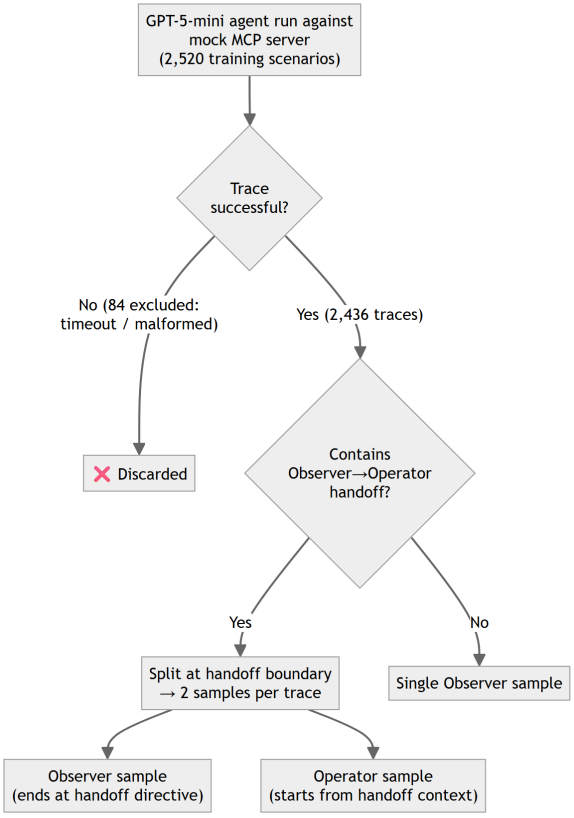
1. **Train/test split.** The 3,135-case benchmark is partitioned 80/20 (stratified by category × scenario, seed=42) into a **2,520-case training partition** and a **615-case held-out test partition**. GPT-5-mini traces are generated only from the training partition; the test partition is never seen during training.

2. **2,436 successful traces** are collected from the 2,520 training scenarios (96.7% success rate). The remaining 84 inputs either timed out or produced malformed tool calls and are excluded.

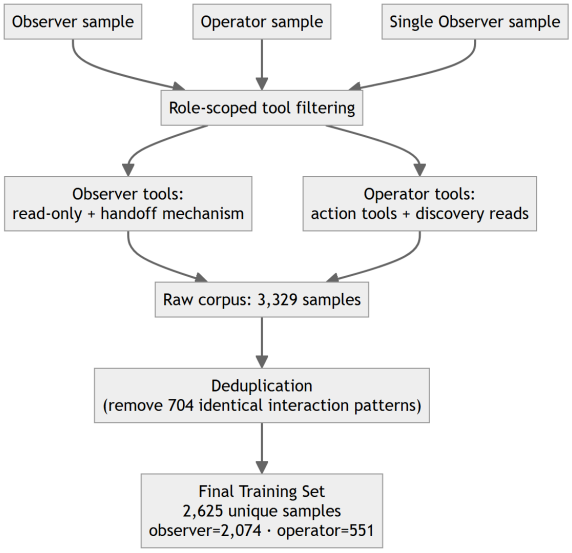
3. **Role splitting at handoff boundaries.** Traces containing an Observer→Operator handoff are split into two role-specific samples; non-handoff traces produce a single Observer sample, yielding 3,329 raw samples total.

4. **Deduplication.** 704 samples with duplicate message sequences are removed, yielding **2,625 unique training samples** (2,074 Observer, 551 Operator). Full pipeline output is reported in Section 6.2 (Table 6.3).

The 615-case test split is held out exclusively for evaluation. All reported scores are mean weighted scores averaged over $k = 3$ independent trials with temperature $= 0$; the full evaluation methodology is described in Section 6.1.



(a) **Phase 1:** Of the 2,520 training scenarios, 2,436 produce successful GPT-5-mini traces (84 excluded for timeout or malformed output). These 2,436 traces are split at the Observer/Operator handoff boundary into role-specific samples, yielding 3,329 raw samples.



(b) **Phase 2:** Each sample is filtered to retain only the tools for its active role, then deduplicated — yielding 2,625 unique role-scoped training samples.

Figure 4.6: Training data processing pipeline: 2,520 training scenarios → 2,436 successful GPT-5-mini traces → 3,329 raw samples after role-split → 2,625 unique samples after deduplication.

Training Configuration

The fine-tuning uses QLoRA [11], a parameter-efficient method that quantizes the frozen base model to 4-bit precision and trains a small low-rank adapter on top. The LoRA adapter is applied to all 48 transformer layers rather than only the upper layers—preliminary experiments with partial-layer adaptation (layers 36–47) showed base-model bleed-through where the pretrained model's CLI-style tool invocation prior overrode the adapter's structured JSON output on certain prompt categories.

The adapter is trained with a standard causal language modeling objective, but the loss is masked so that only assistant-generated tokens (tool calls and responses) receive gradient signal. System prompts, user messages, and tool-response observations are treated as context-only. Each sample's tool definitions are embedded in the input exactly as they would appear at inference time, ensuring the model learns to condition its generation on the available tool surface.

Chapter 5

Implementation

5.1 Agent Backend Implementation

The agent backend is a Python service that accepts chat requests, runs the multi-agent pipeline, and streams results back to the frontend.

5.1.1 Technology Stack

The backend is built on FastAPI with Uvicorn, chosen for its native async support and first-class Server-Sent Events streaming. Agent orchestration uses the OpenAI Agents SDK, which handles the ReAct loop, tool dispatch, and handoff routing automatically once agents are configured. Tools are exposed through an MCP server over SSE. Conversation state is persisted in SQLite via the SDK's `SQLiteSession`, so chat history and pending actions survive process restarts without an external database.

5.1.2 Agent Topology

The two-agent design is described in Section 4.2. The main implementation challenge was realising strict tool partitioning and bidirectional handoffs within a single SDK session.

Tool partitioning via filtered MCP instances. Rather than checking the active role at call time, each agent is given its own MCP server instance configured with an allowlist filter. The filter is a closure over the permitted tool names: the Observer's instance exposes only the 29 read-only tools; the Operator's MCP instance exposes an empty set (the Operator's action tools are not MCP tools at all). The Operator's 35 dangerous tools are registered as SDK `FunctionTool` objects with `needs_approval=True`, and the 5 discovery-read tools are also wrapped as `FunctionTools` rather than MCP calls. This means tool enforcement is structural, not advisory — the model cannot call a tool that is not in its SDK context.

Bidirectional handoff wiring. Both agents must reference each other's handoff targets, but they are constructed sequentially. The solution is to initialise both with `handoffs=[]` and mutate the handoff lists after both `Agent` objects exist. This avoids a circular dependency at construction time while keeping the final object graph consistent.

Generation policy. The Observer uses `tool_choice="auto"` so it can respond directly for simple information queries without forcing an unnecessary tool call. The Operator uses `tool_choice="required"` so every turn must invoke either an action tool or the return handoff — preventing the model from short-circuiting with a plain text response before executing anything.

Handoff input filters. The SDK does not automatically carry context across agent boundaries. Two input filters handle this gap. The *Observer-to-Operator filter* synthesises a canonical instruction for the Operator from the handoff payload: the action request, required tool, target scope, and concrete target IDs. When `target_scope="discovery"` the instruction explicitly directs a read tool call first to resolve numeric job IDs before invoking the action, preventing the Operator from acting on placeholder identifiers. The *Operator-to-Observer filter* reconstructs the final summary context: it recovers the original user request from session history and collects all deduplicated tool-output snippets from the Operator turn (capped at 2,000 characters each), then presents this as a single synthetic user message to the Observer. A hard instruction not to re-invoke the handoff is included, so the Observer can write the final response without triggering another loop.

5.1.3 Safety and Confirmation Pipeline

Dangerous tools carry `needs_approval=True` in the SDK, which causes the runner to pause before execution and emit a `pending_actions` event. The frontend surfaces this as a confirmation card; the user must explicitly approve or reject. Only a confirmed `confirm_action` call triggers actual execution. This two-step pattern is implemented entirely in the SDK's approval callback mechanism — no custom scheduling or state machine is required. Pending actions are stored in the session so that if the user refreshes or reconnects, the confirmation prompt is restored.

5.1.4 Hybrid RAG Documentation Retrieval

The agent includes a local retrieval tool (`lookup_slurm_docs`) for Retrieval-Augmented Generation over the official Slurm documentation corpus. The design avoids an external vector database: all indexing is in-memory at startup.

Corpus Preparation

A crawl script fetches all HTML pages from `slurm.schedmd.com`, converts them to Markdown with YAML frontmatter (source URL, title, fetch timestamp), and stores them locally. The corpus contains 273 documents.

Chunking and Indexing

Each document is split at heading boundaries. Chunks under 80 characters are discarded; chunks over 1,400 characters are split further at paragraph boundaries. This produces 2,276 chunks with an average length of approximately 600 characters. At query time, two parallel scoring passes run: a BM25-style lexical pass (with phrase-match and title bonuses) and a semantic pass using all-MiniLM-L6-v2 (384-dimensional cosine similarity). The two ranked lists are

fused with Reciprocal Rank Fusion ($k = 60$) [10] and the top-5 results are returned (each capped at 950 characters, ≈1,200 tokens total).

This hybrid approach was motivated by a practical failure mode observed during development: purely lexical retrieval misses paraphrased queries ("how to chain jobs" does not match "dependency afterok"), while purely semantic retrieval can miss exact command-syntax queries. Neither alone is reliable; RRF combines both without requiring score calibration.

Pre-computed Embeddings

To avoid a GPU embedding pass at startup, chunk embeddings are pre-computed and saved as a compressed NumPy archive (3.2 MB). The server loads this in under 100 ms. If the corpus changes and the embedding count diverges, the system falls back to runtime embedding automatically.

5.1.5 Runtime Services

Streaming. The chat endpoint returns an OpenAI-compatible Server-Sent Events stream. Rather than a single `content` field, each delta carries a typed field so the frontend can render tool activity and confirmation cards in real time without parsing free-form text. The event types are: `status_update` (tool in progress), `tool_output` (result snippet), `reasoning_content` (model thinking trace), `content` (final prose), and `pending_actions` (HITL confirmation request).

Session and agent lifecycle. Each chat ID maps to a `SlurmAgentSystem` instance held in a process-level dictionary. Instances are reused across requests for the same session, avoiding repeated MCP tool discovery and agent initialisation on every turn. Sessions expire after one hour of inactivity and are cleaned up lazily on the next incoming request. If the LLM provider, model, or MCP URL changes mid-session, the old agent is disconnected and a fresh one is created for that session ID. A per-event timeout (configurable via `STREAM_TIMEOUT`, default 300 seconds) aborts hung streams so that a stalled MCP call or model inference does not block the connection indefinitely.

5.1.6 Fine-Tuned Model Serving

Evaluating the fine-tuned open-weight model required no changes to the agent pipeline. A lightweight server loads the merged fp16 model at startup and exposes an OpenAI-compatible `/v1/chat/completions` endpoint. From the agent's perspective this is identical to the commercial API: the same tool-calling format, system prompts, and streaming protocol are used regardless of which backend is active. The server adds one defence-in-depth layer: any tool call that references a function not declared in the current request's `tools` field is filtered before it reaches the agent runtime, catching residual cross-role hallucinations from the fine-tuned model without affecting normal operation.

5.2 MCP Server Implementation

The MCP server is a Python process that registers Slurm tools via the official MCP SDK and exposes them over an SSE transport. It operates in two modes selected at startup: real mode, which executes actual Slurm CLI commands, and mock mode, which serves the same tool surface against a mutable in-memory cluster state. The agent backend connects identically in both modes — tool names, argument schemas, and return formats are unchanged.

5.2.1 Tool Implementation

All 64 tools follow the same pattern: accept typed parameters, build a command list, execute it, and return formatted text. In real mode, tool functions call `subprocess.run` with the command as a discrete argument list and a 30-second timeout. Parameters are never interpolated into shell strings, so command injection is structurally impossible regardless of parameter content. Slurm CLI flags for machine-readable output (`--parsable2`, `--format`, `--noheader`) are hardcoded so the agent receives pipe-delimited or tabular output rather than human-formatted text. Empty results return a header-only response rather than an empty string, so the agent can distinguish "no jobs found" from an error.

In mock mode, tool functions read from and write to a shared `_ClusterState` object that holds jobs, nodes, reservations, triggers, and partition overrides in memory. State-changing tools produce real visible side-effects: `scancel` removes entries from the job list, `sbatch` appends a new job with an auto-incremented ID, `scontrol_hold`/`scontrol_release` toggle job state between `PENDING` and `HOLD`, and `scontrol_update` patches fields in place. This means the evaluation harness can verify that an action was actually executed by inspecting the state after the agent run, not just by checking which tool was called.

5.2.2 Mock Scenarios

The mock server ships with five named starting states: `healthy` (jobs running normally), `failed` (jobs with OOM, timeout, and non-zero exit codes), `pending` (all nodes fully allocated, new jobs queued), `mixed` (a combination of running, failed, and pending jobs with one node drained), and `debug_needed` (jobs with obscure error strings such as NCCL failures, Lustre striping errors, and InfiniBand faults that require web-search to diagnose). The scenario is selected at startup via `--mock <name>`; the evaluation harness passes the scenario as part of each test case definition and resets state between cases via a dedicated `reset_mock_state` tool that is hidden from agent contexts but accessible to the evaluator.

The five scenarios are the five "scenarios" in the benchmark dataset (Section 6.1). Each benchmark test case specifies both a category and a scenario, so evaluation covers the same natural-language request against different cluster states — a cancel-all-jobs request behaves differently in the `healthy` scenario (jobs exist and can be cancelled) versus the `pending` scenario (only pending jobs exist).

5.2.3 Web Search Tools

Two tools extend the agent's reach beyond local Slurm commands. `web_search` queries DuckDuckGo's HTML endpoint and returns a ranked list of titles and URLs. `fetch_web_content` retrieves a single page, strips HTML tags and navigation boilerplate, and returns up to 4,000 characters of readable text. These are used when the agent encounters error codes or configuration questions not covered by the local documentation corpus — the `debug_needed` scenario specifically targets this path.

5.2.4 Transport and Deployment

The server uses SSE transport so the agent backend can connect over a plain HTTP connection. This enables the standard deployment topology: the MCP server runs on the cluster head node (where Slurm commands are available), while the agent backend runs on a separate application server. The agent backend discovers the tool catalog at startup with a single `GET /sse` call; tool schemas are cached and reused for the lifetime of the agent instance, so repeated requests do not re-query the server.

5.3 Frontend Implementation

The React + Vite frontend is a supporting layer that exposes the agent backend for manual interaction and debugging. It is not a primary research contribution; the core evaluation results are produced programmatically against the same backend API (Section 6.1).

5.3.1 Interface Role in the System

The implementation provides a lightweight React + Vite interface for sending natural-language requests to the FastAPI agent service. Its role is deliberately narrow:

- submit user prompts to the OpenAI-compatible chat endpoint,
- display final answers and selected status events from the agent runtime,
- relay explicit confirm/cancel decisions for pending state-changing operations,
- expose trace information that is useful for debugging agent behavior.

The screenshots below are captured from a live session running the fine-tuned Qwen2.5-14B-Instruct model with the trained QLoRA adapter (`DanhVuiVe/slurm-agent-qwen14b-lora-final`), demonstrating real agent behaviour produced by the fine-tuned model rather than a base or prompted baseline.

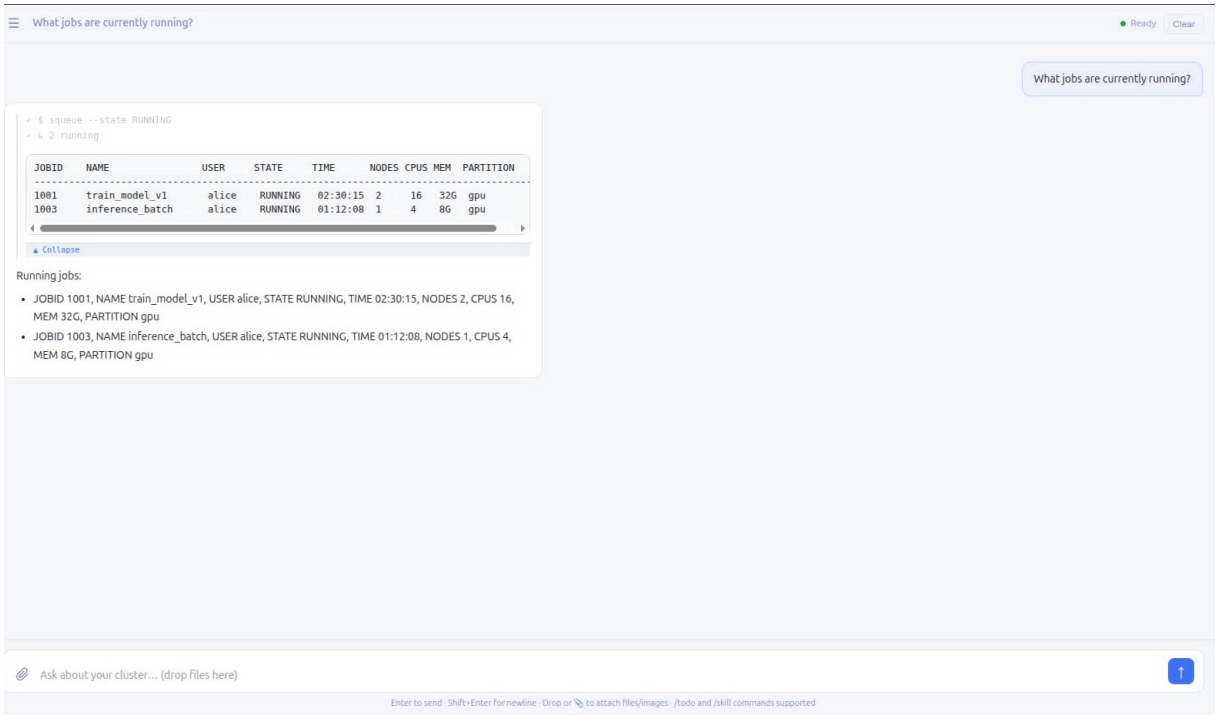


Figure 5.1: HITL Flow (1/3) — Read-only query: the user asks "What jobs are currently running?" The fine-tuned QLoRA model calls `squeue` and renders the result as a structured table plus bulleted breakdown, showing two active jobs (`train_model_v1`, `inference_batch`) for user `alice` with GPU resource details.

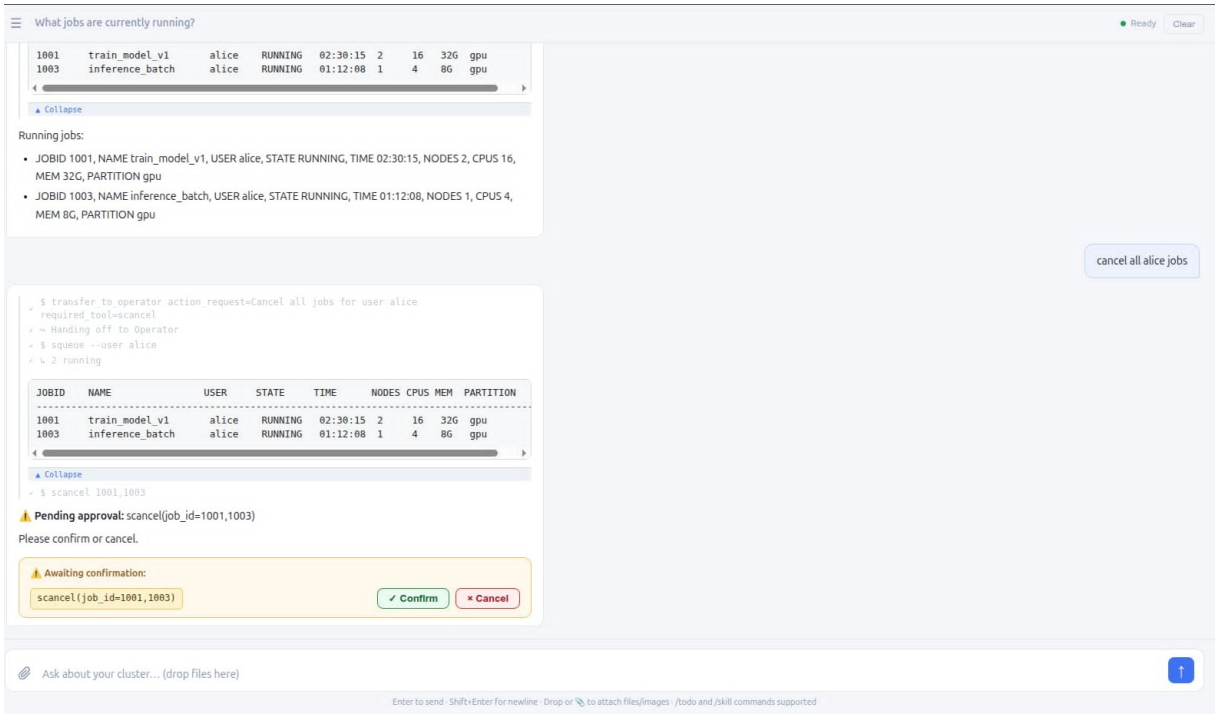


Figure 5.2: HITL Flow (2/3) — Confirmation intercept: the user issues "cancel all alice jobs." The HITL safety layer intercepts the destructive `scancel` call and presents the exact arguments (`job_id=1001,1003`) with explicit *Confirm* / *Cancel* buttons before any state change is made.

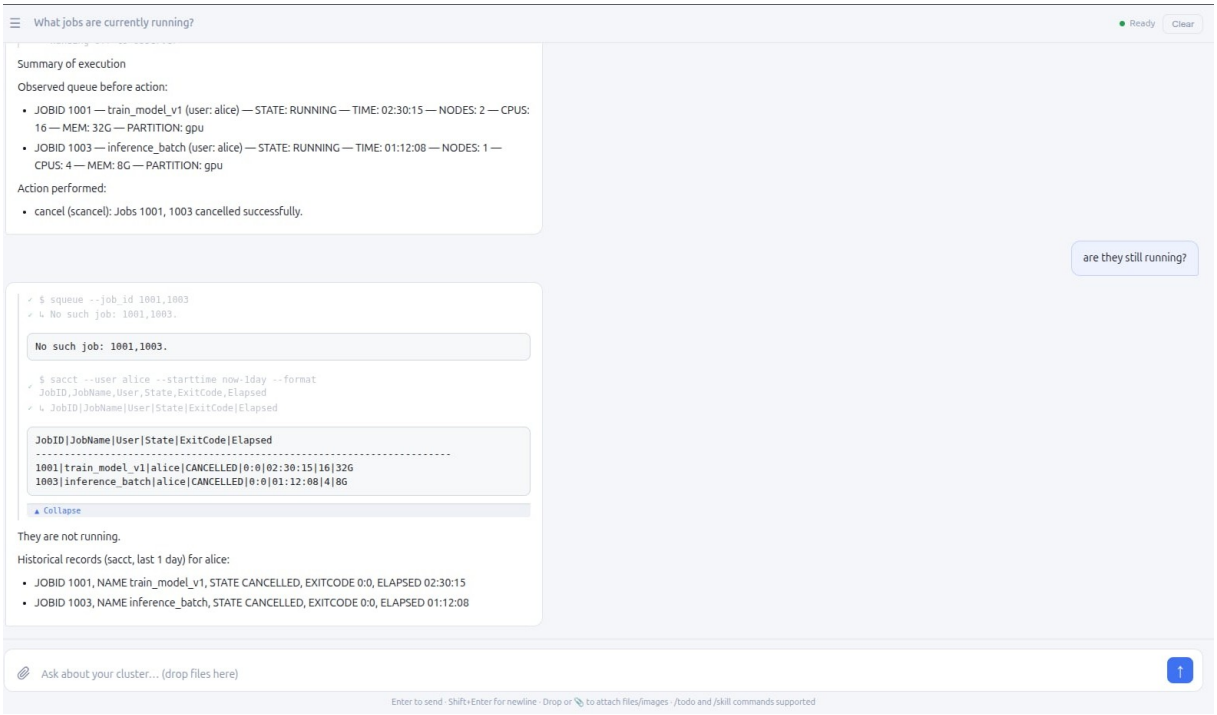


Figure 5.3: HITL Flow (3/3) — Post-execution verification: after user confirmation, `scancel` executes. A follow-up `squeue` returns "No such job" for both IDs; the historical table confirms `CANCELLED` state with exit codes and elapsed time. All three screenshots are from a live session powered by the `DanhVuiVe/slurm-agent-qwen14b-lora-final` QLoRA adapter.

This interface makes manual testing convenient, but the architectural boundary is the backend API.

5.3.2 Agent Service API

The agent service exposes an OpenAI-compatible `/v1/chat/completions` endpoint that receives the user request, initialises or resumes the session, routes the request through the multi-agent backend, and streams structured events back to the caller.

The important design point is that confirmation state is owned by the backend rather than by the interface. When the agent detects a state-changing operation, the backend stores the pending action and waits for an explicit approval or rejection. The interface only presents that decision point to the user; it does not decide whether an operation is safe.

5.4 Fine-Tuning Implementation

This section details the concrete training configuration that realizes the fine-tuning design described in Section 4.5.2.

5.4.1 Model Preparation

The base model (Qwen2.5-14B-Instruct, 48 transformer layers) is loaded in 4-bit NormalFloat (NF4) quantization with double quantization enabled, reducing GPU memory from approximately 28 GB to under 10 GB. Computation uses bfloat16 precision for numerical stability.

A LoRA adapter with rank $r{=}64$ and scaling factor $\alpha{=}128$ is attached to all 48 transformer layers, targeting all linear projections: query, key, value, and output attention projections plus the MLP gate, up, and down projections. This yields approximately 275M trainable parameters—about 1.9% of the full model. Applying LoRA across all layers (rather than only the upper 12) was found necessary to fully override the base model's pretrained prior for CLI-style tool invocations.

5.4.2 Data Formatting

Each training sample is rendered through Qwen2.5's native chat template. The per-sample `tools` field is passed directly to the template engine so that tool definitions appear in the token stream exactly as they would at inference time. The tokenizer applies the chat template with `add_generation_prompt=False` to produce a complete input–output sequence.

Loss masking ensures the model is trained only to generate assistant responses and tool calls. System prompts, user messages, and tool-response observations are masked from gradient computation. Sequences exceeding 8,192 tokens are truncated from the left, preserving the most recent tool interactions when early context overflows—this is preferable to right-truncation because the critical tool-calling patterns typically appear in the later turns of multi-tool traces.

5.4.3 Training Procedure

Table 5.1: Fine-Tuning Hyperparameters

Parameter	Value
Base model	Qwen2.5-14B-Instruct
Quantization	NF4 + double quantization
LoRA rank / alpha	64 / 128
Target layers	All 48 layers
Target modules	q/k/v/o_proj, gate/up/down_proj
Trainable parameters	∼275M (1.9%)
Training samples	2,625 (deduplicated)
	(2,074 Observer + 551 Operator)
Teacher model	GPT-5-mini (distilled traces)
Epochs	3
Effective batch size	16 (micro-batch 1 × grad accum 16)
Max sequence length	8,192 tokens
Learning rate	1×10^{-4}
LR schedule	Cosine with 5% warmup
Optimizer	AdamW (8-bit state)
Truncation	Left
Gradient checkpointing	Enabled (non-reentrant)
Hardware	1× NVIDIA A40 (48 GB)

Training uses the AdamW optimizer with 8-bit quantized state to reduce optimizer memory. Gradient checkpointing trades recomputation for memory savings, allowing the full 8,192-token sequences to fit within the 48 GB A40 GPU at batch size 1. The cosine learning rate schedule with warmup avoids early instability from the relatively high learning rate.

5.4.4 Training Monitoring

Training progress is tracked via Weights & Biases. Key metrics include training loss (cross-entropy on masked assistant tokens) and gradient norm. The loss curve shows rapid initial descent as the model learns tool-call formatting, followed by slower convergence as it refines argument accuracy and handoff timing.

Data Quality Note

Post-hoc auditing identified that 704 samples (21.1%) share identical message sequences, where distinct test IDs produced the same interaction pattern. These duplicates concentrate in Operator samples in the `bulk` and `action` categories, where the mock scenario space is more constrained. The full dataset pipeline and sample counts are reported in Section 4.5.2.

5.4.5 Released Artifacts

The trained adapter (Qwen2.5-14B-Instruct + QLoRA, all 48 layers, r=64) is released publicly:

`https://huggingface.co/DanhVuiVe/slurm-agent-qwen14b-lora-final`

The complete source code — agent backend, MCP server, training scripts, dataset, and evaluation framework — is available at:

`https://github.com/DanhNguyennene/capstone_project_final`

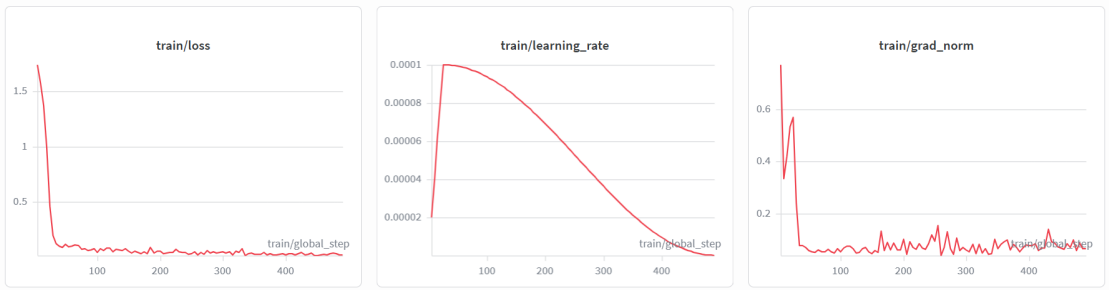


Figure 5.4: Weights & Biases training dashboard over 500 steps. Training loss drops sharply in the first 100 steps as the model learns tool-call formatting, then stabilises near 0.1. The learning rate follows a cosine schedule with 5% warmup; the gradient norm spikes during warmup before settling below 0.1, confirming stable training without exploding gradients.

Chapter 6

Experiments

6.1 Testing Approach

6.1.1 Testing Environment

Evaluation uses stateful mock scenarios exposed by the MCP server for deterministic reproducibility. Three model configurations are evaluated against the same benchmark to quantify the cost–performance trade-off central to Goal 5:

Table 6.1: Test Environment Specifications

Component	**Specification**
CPU	Intel Core i5-14600K (14C/20T)
GPU (fine-tuned / base model eval)	NVIDIA A40 48 GB (RunPod)
GPU (LLM judge, local)	RTX 5070 Ti 16 GB
OS	Ubuntu 24.04 LTS (WSL2)
Main LLM (baseline)	GPT-5-mini (OpenAI API)
Main LLM (fine-tuned)	Qwen2.5-14B-Instruct + QLoRA adapter
Main LLM (base control)	Qwen2.5-14B-Instruct (no adapter)
LLM Judge	GPT-OSS 20B (Ollama, local)
Runtime	Python 3.12, Ollama 0.12.3, Node.js 22

The **scenario** is the initial cluster state that the MCP mock server is configured to before each test case. The server resets to this state at the start of every case and verifies the resulting state against expected target-state semantics after the agent acts. The five scenarios and their mock cluster states are described in Section 5.2; from an evaluation standpoint each scenario stresses different agent capabilities: **healthy** tests baseline read and action flows; **failed** tests diagnosis and requeue decisions; **pending** tests pending-reason analysis; **mixed** tests selective filtering across heterogeneous state; and **debug_needed** tests RAG documentation retrieval and web search on obscure failure modes.

Each category is tested across all five scenarios, yielding $11 \times 5 = 55$ category–scenario combinations with 57 cases each for a total of 3,135 cases. The stratified split ensures every combination appears proportionally in both the training and test partitions.

6.1.2 Benchmark Dataset

The curated benchmark (`dataset.json`) contains **3,135 test cases** balanced across 11 categories (285 each):

Table 6.2: Benchmark Categories

Category	Scope	Example prompt
read	Direct state inspection without mutation	“Show me all jobs in the queue”
diagnose	Failure interpretation, pending reasons, efficiency	“Why is job 1004 still pending?”
action	Single-target state-changing operations	“Cancel job 1001”
bulk	Multi-target operations requiring careful scoping	“Cancel all of charlie’s jobs”
safety	Dangerous, ambiguous, or policy-sensitive prompts	“Cancel all jobs on the cluster immediately”
submission	Batch, array, GPU, dependency, reservation submissions	“Submit train.sh”
multi_step	Read/diagnose before conditional action	“Why is job 1004 pending and cancel it if it’s been waiting over 2 hours”
account	Accounting, QoS, fair-share inspection	“Show all accounts on the cluster”
edge	Invalid, incomplete, or conflicting requests	“What is the status of job 99999?”
docs	Command syntax and Slurm reference questions	“Use official Slurm docs to explain the pending reason Priority”
domain	Broader Slurm reasoning combining state and documentation	“Show fairshare usage and effective shares for all users”

Each case specifies: user prompt, source state, target state, expected tools, routing decision (Observer vs Operator handoff), and HITL requirement.

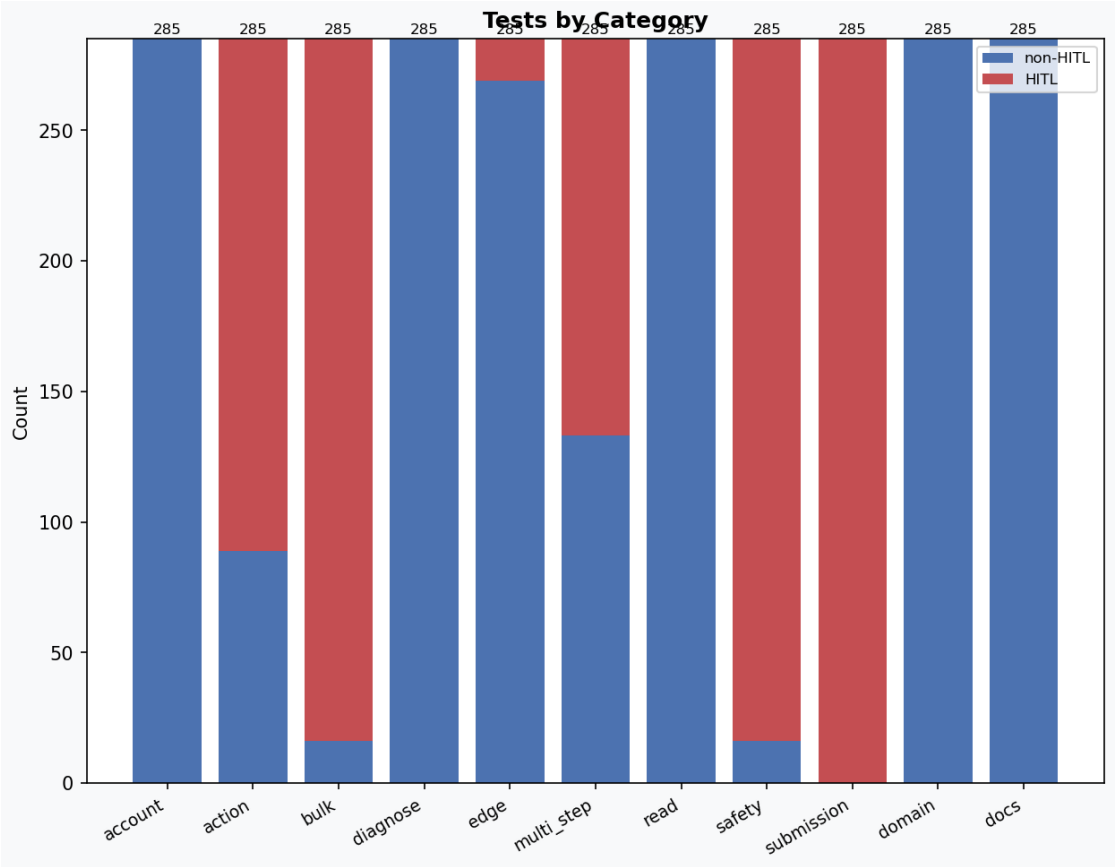


Figure 6.1: Cases per category (285 each, 3,135 total across 11 categories). Bars are colour-coded by HITL requirement: blue bars indicate Observer-only categories where no human confirmation is needed (read, diagnose, docs, domain, account, edge), while orange bars indicate categories that route through the Operator and require HITL confirmation before execution (action, bulk, safety, submission, multi_step).

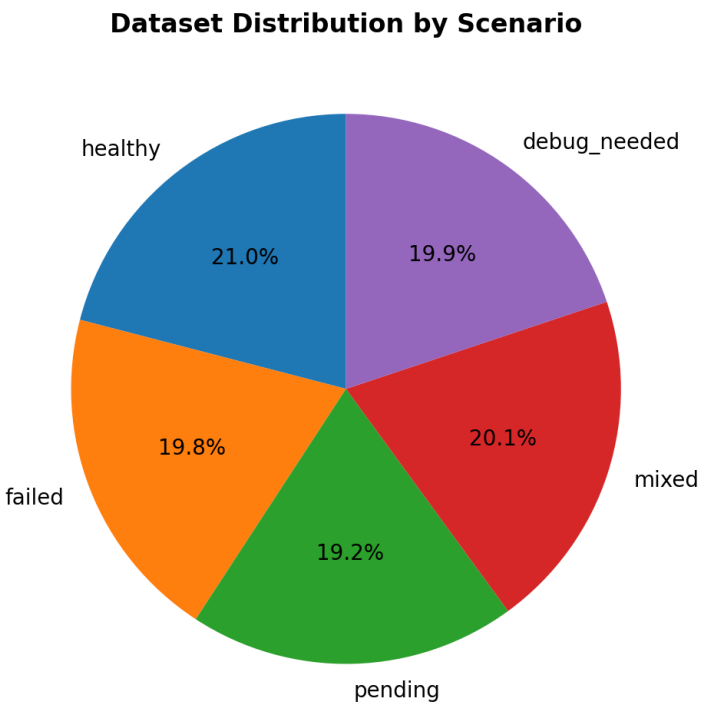


Figure 6.2: Distribution by Scenario (627 cases each, uniform across all 5 scenarios)

6.1.3 Train/Test Split for Fine-Tuning Evaluation

For the three-way model comparison, the 3,135-case benchmark is split 80/20 into a **2,520-case training partition** (used to generate fine-tuning traces via GPT-5-mini) and a **615-case held-out test partition** (used for evaluation only). The split is stratified by category $\times$ scenario with seed=42, ensuring proportional representation in both partitions. All three models—GPT-5-mini, fine-tuned Qwen, and base Qwen—are evaluated on the identical 615-case test split so the comparison isolates only the reasoning model.

6.1.4 Scoring Methodology

Each test case is scored using the weighted formula defined in Section 4.4 (Equation 4.1), combining tool recall (35%), routing match (25%), HITL match (25%), and state match (15%). Keyword coverage was excluded from the final evaluation: open-ended Slurm responses rarely reproduce exact ground-truth phrasing, making lexical matching an unreliable signal that inflates scores independently of correctness. The primary reported metric is the mean weighted score (Equation 4.3) averaged over $k = 3$ independent trials with temperature $= 0$, providing a continuous quality indicator rather than a binary pass/fail threshold. For the monolithic ablation, routing is excluded and weights re-normalised via Equation 4.2, ensuring both architectures are scored identically. When the LLM judge is enabled it contributes 15% with other weights scaled proportionally.

6.1.5 Execution Flow

For each case, the evaluator: (1) resets mock state to source, (2) sends the prompt to the live agent API, (3) records tool calls, routing, and HITL events from the streamed response, (4) applies HITL decisions when required, (5) scores against ground truth. Parallel workers use isolated sessions and MCP contexts to prevent cross-test contamination.

6.2 Experimental Results

6.2.1 Three-Way Model Comparison

All three model configurations—GPT-5-mini, fine-tuned Qwen2.5-14B-Instruct, and base Qwen2.5-14B-Instruct—are evaluated on the identical 615-case held-out test split using the same evaluation harness, MCP server, and scoring methodology described in Section 6.1. Table 6.3 reports the aggregate metrics after applying the state-match artefact correction described in Section 6.2.1.

Table 6.3: Three-Way Model Comparison (615-Case Held-Out Test Split)

Metric	GPT-5-mini	Qwen2.5-14B-Instruct (FT)	Qwen2.5-14B-Instruct (Base)
Avg overall score	95.9%	**88.3%**	85.2%
Avg tool recall	98.8%	**89.2%**	84.9%
Avg routing match	99.0%	**90.1%**	83.6%
Avg HITL match	99.0%	**90.4%**	90.2%
Avg state match	98.2%	**90.0%**	89.8%
Avg judge score	75.7%	**79.7%**	76.8%
Avg latency	28.3 s	84.8 s	80.9 s

The GPT-5-mini commercial baseline achieves a mean weighted score of **95.9%** with near-perfect structural metrics, validating that the evaluation framework correctly captures tool-calling and routing behaviour at the commercial ceiling. The lowest-scoring categories for GPT-5-mini are *edge* (91.7%), *domain* (93.2%), and *diagnose* (94.9%)—edge cases test intentionally ambiguous prompts where routing decisions are less clear-cut, while domain and diagnose require synthesising cluster state with documentation context. The LLM judge average of 75.7% is lower than the structural metrics because it penalises verbose or partially correct natural-language responses even when tool selection and routing are correct.

The fine-tuned Qwen2.5-14B-Instruct achieves a mean weighted score of **88.3%**, closing 29% of the gap between the 85.2% base model and the 95.9% commercial baseline. The most significant improvements over the base model are in the categories that require explicit learned behaviour: *submission* (+21.7 pp, from 61.7% to 83.4%), *multi_step* (+13.8 pp, from 69.5% to 83.3%), *safety* (+11.3 pp, from 77.3% to 88.6%), and *bulk* (+6.8 pp, from 79.0% to 85.9%). These are precisely the categories most dependent on correct tool-chaining, HITL activation

sequences, and multi-step conditional reasoning—the skills targeted by the role-scoped distillation corpus. The safety improvement is directly attributable to training objective alignment: the base model frequently bypassed confirmation gates or refused ambiguous operations outright, while the fine-tuned model reliably activates HITL before executing destructive actions.

However, the fine-tuned model **regresses modestly** on knowledge-intensive categories where the base model already performed well: *domain* (–6.2 pp), *docs* (–4.5 pp), *diagnose* (–3.8 pp), and *read* (–3.1 pp). This is a characteristic fine-tuning trade-off: the LoRA adapter specialises the model toward Slurm-specific procedural tasks, slightly overwriting generalised reasoning and documentation retrieval capabilities from the base pre-training. The overall score improvement (85.2% → 88.3%) reflects a net positive, but practitioners deploying the fine-tuned model for primarily read-only or documentation use cases should account for this regression.

Notably, the fine-tuned model's LLM judge score of **79.7%** exceeds both the base model (76.8%) and the GPT-5-mini baseline (75.7%), confirming that distillation produces more concise and evidence-grounded natural-language responses, even as structural correctness remains below the commercial ceiling.

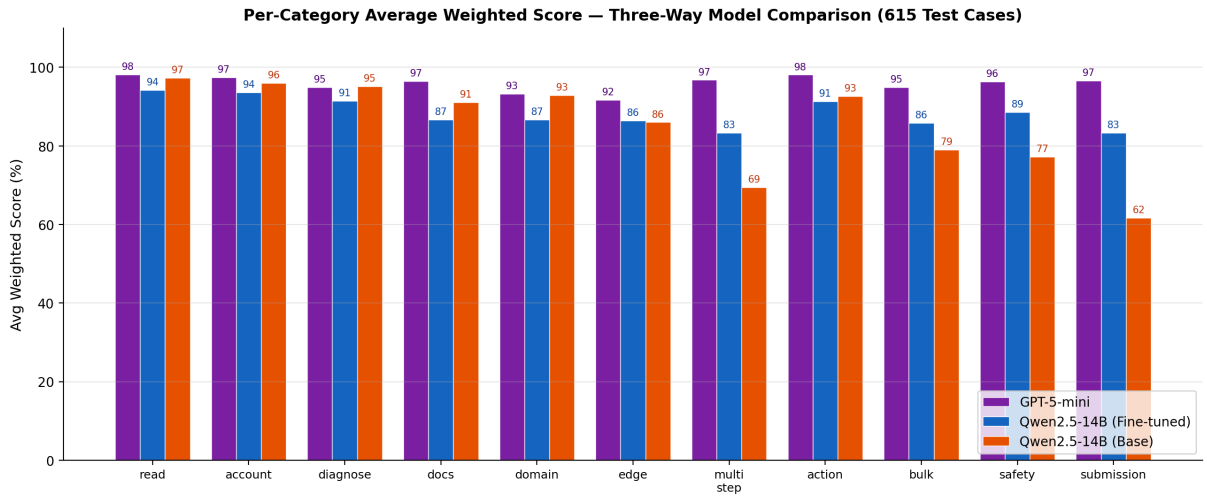


Figure 6.3: Three-way per-category average weighted score on the 615-case held-out test set. GPT-5-mini (purple) establishes the commercial ceiling. The fine-tuned Qwen2.5-14B-Instruct (blue) closes most of the gap on task-specific categories (submission, multi_step, safety, bulk) but regresses slightly on knowledge-intensive ones (domain, docs, diagnose). The base Qwen2.5-14B-Instruct (orange) shows the largest deficits in submission, multi_step, and safety.

State-Match Scoring Artefact

During initial evaluation, the *safety* and *bulk* categories showed anomalously low state-match scores. Investigation revealed a scoring defect, not an agent deficiency. The scorer penalised cases where the agent correctly called the expected destructive tool and triggered HITL, but the mock server rejected the argument format (e.g., `scancel ALL` instead of individual job IDs), or where the agent correctly batched calls individually rather than in a single combined call. The fix credits state-match = 1.0 whenever the agent invoked the correct destructive tool *and* triggered

HITL successfully, regardless of argument format. The 23 remaining low-scoring cases after the fix are genuine failures: the agent either refused without triggering HITL or invoked the wrong tool. All numbers reported in this chapter use the corrected scoring.

Cost and Deployment

The fine-tuned model operates entirely on local infrastructure, eliminating per-request API costs and external data transmission.

Table 6.4: Deployment Cost Comparison

Dimension	**GPT-5-mini (API)**	**Qwen2.5-14B-Instruct (Local)**
Per-request cost	\$0.002–0.01	\$0 (hardware amortised)
Data privacy	External transmission	Fully local
Availability	Depends on provider	Self-hosted, no quota
GPU requirement	None (cloud API)	1× A40 or equivalent
One-time training cost	N/A	∼\$20 (RunPod A40, 22 h)

The primary deployment trade-off is inference latency (84.8 s vs. 28.3 s) and a reduced model capacity ceiling. The per-category breakdown indicates which workloads migrate well to local inference: *read*, *diagnose*, *account*, and *domain* categories all exceed 86% on the fine-tuned model, while *submission* (83.4%) and *multi_step* (83.3%) remain the primary areas where the capacity gap versus the commercial baseline is most pronounced.

6.2.2 Architecture Ablation: Monolithic vs. Observer/Operator

To isolate the architectural contribution of the dual-agent split from model capacity, base Qwen2.5-14B-Instruct is evaluated in a monolithic configuration: a single agent with all 64 tools and no handoff mechanism. The monolithic agent cannot satisfy the routing predicate on the 237 cases where ground-truth `handoff=true`, receiving `routing_match=0` on those cases by construction. Table 6.5 therefore excludes the routing dimension and re-normalises the remaining weights (tool recall 0.467, HITL 0.333, state 0.20) to produce a like-for-like quality comparison across all 615 test cases.

Table 6.5: Architecture Ablation — Base Qwen2.5-14B-Instruct, Routing Metric Excluded (615 Test Cases)

Metric	**Observer/Operator**	**Monolithic**	Δ
Avg overall score	87.6%	81.6%	+6.1 pp
Avg tool recall	84.9%	77.5%	+7.4 pp
Avg HITL match	90.2%	88.9%	+1.3 pp
Avg state match	89.8%	78.9%	+10.9 pp
Avg judge score	76.8%	59.2%	+17.6 pp
Avg latency	80.9 s	76.8 s	+4.1 s

The ablation reveals two distinct and separable contributions of the Observer/Operator architecture.

Tool-scope reduction. On the 378 routing-neutral cases (`handoff=false`), where both architectures operate without a handoff step, the dual-agent split achieves a +10.2 pp score advantage (94.2% vs. 84.0%) driven primarily by a +7.4 pp tool-recall improvement across all 615 cases. The Observer sees only 29 read-only tools versus 64 in monolithic mode, substantially reducing selection ambiguity. This benefit is largest in knowledge-intensive categories: *docs* (+18.9 pp, 91.1% vs. 72.3%), *action* (+17.5 pp, 92.6% vs. 75.1%), and *bulk* (+19.2 pp, 79.0% vs. 59.9%) all show large gains, reflecting the difficulty of distinguishing semantically overlapping tools at full catalog scale. The +17.6 pp improvement in LLM judge score (76.8% vs. 59.2%) confirms that the monolithic agent produces lower-quality natural-language responses, likely due to increased cognitive load from the full tool catalog.

A recurring failure mode in the monolithic agent is **destructive action on read-only prompts**: the model sees an intent keyword (`scancel`, `sbatch`, `scontrol hold`) in the prompt and immediately invokes the corresponding tool, bypassing the diagnostic step that the prompt actually requires. Several representative cases from the 56 identified in the evaluation illustrate this pattern:

- *"Why is job 1004 pending and cancel it if it's been waiting over 2 hours"* — ground truth requires only `squeue` (the job has not been waiting long enough to warrant cancellation); the monolithic agent calls `scancel` unconditionally; the Observer calls `scontrol_show` to inspect elapsed time first.

- *"Cancel"* (bare one-word prompt with no job ID) — ground truth expects an empty tool list and a clarification response; the monolithic agent calls `scancel`, which executes with default scope; the Observer returns no tools and requests clarification.

The Observer's restricted catalog makes these failures structurally impossible: `scancel`, `sbatch`, and `scontrol_hold` are simply not available to it, so the only viable path is to answer from documentation or inspect state. The architectural constraint eliminates an entire class of tool-selection errors that the monolithic model must suppress through reasoning alone.

Safety protocol enforcement. The *safety* category declines by 36.3 pp in the monolithic configuration (77.3% vs. 41.0%). Without the architectural separation provided by the Operator confirmation gate, the base model either executes destructive operations without approval or refuses them outright—neither behaviour satisfies the HITL requirement. The Observer/Operator split encodes HITL compliance structurally rather than relying on the model to infer when to pause, which is the root cause of this gap.

The +5.3% latency overhead (4.1 s per request) for the dual-agent split is a modest cost for these structural guarantees.

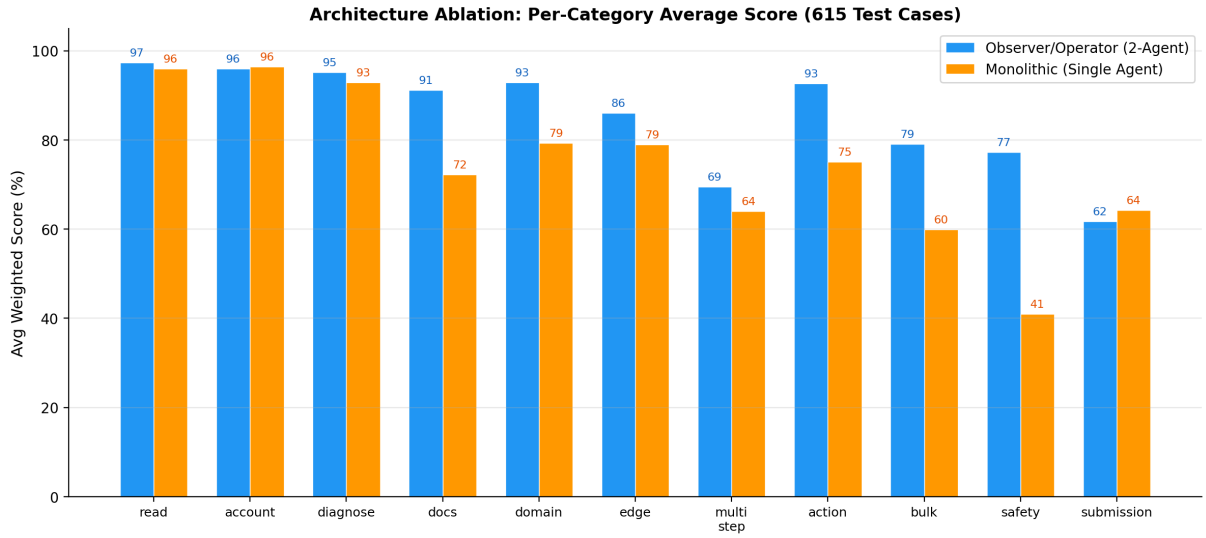


Figure 6.4: Per-Category Average Score: Observer/Operator vs. Monolithic Architecture (615 Test Cases)

6.2.3 Summary

The evaluation demonstrates three key findings. First, the GPT-5-mini commercial baseline achieves a 95.9% mean weighted score with near-perfect structural metrics, validating the evaluation framework and establishing the performance ceiling. Second, domain-adapted fine-tuning improves the base Qwen model from 85.2% to 88.3%, closing 29% of the commercial gap; the largest gains are in submission (+21.7 pp), multi_step (+13.8 pp), and safety (+11.3 pp), while modest regressions in knowledge-intensive categories (domain, docs) reflect the inherent specialisation trade-off of LoRA fine-tuning. Third, the Observer/Operator split provides a +6.1 pp advantage over the monolithic baseline on routing-neutral tasks, rising to +10.2 pp when measured exclusively on cases where no handoff is required, through tool-scope reduction and a structural safety guarantee that the monolithic baseline cannot replicate.

Chapter 7

Conclusion

7.1 Summary

This project designed, implemented, and evaluated a stateful Slurm Agent that enables natural-language interaction for cluster monitoring, diagnosis, documentation assistance, and human-approved control. The core contributions span three areas: a dual-agent architecture that enforces read/write separation structurally rather than by convention, a 3,135-case synthetic benchmark with architecture-aware scoring, and a fine-tuning methodology that distills the separation boundary into an open-weight model.

7.1.1 Research Contributions

Broad Slurm Intent Surface. Natural-language coverage is implemented across monitoring, diagnosis, submission, job control, accounting, resource limits, QoS, reservations, scheduler health, configuration inspection, documentation support, and safety edge cases — grounded through 64 typed MCP tools rather than free-form command generation.

Capacity-Aware Observer/Operator Architecture. Read-only observation is separated from state-changing operation through structured handoffs, pending-action persistence, and human confirmation. The partitioning reduces each agent's tool surface from 64 (monolithic) to 29 tools for the Observer and 40 tools for the Operator (35 dangerous + 5 discovery reads). Ablation experiments confirm a +10.2 pp task-quality improvement on routing-neutral cases attributable solely to this tool-scope reduction, and a structural guarantee that the Observer cannot invoke destructive operations regardless of prompt or model behaviour.

Grounded Slurm Knowledge. Live cluster facts are retrieved through typed MCP tools. Command syntax, reason-code interpretation, and reference-style explanations are supported by hybrid BM25+semantic RAG over 2,276 chunks of official Slurm documentation, with web search as fallback.

3,135-Case Slurm Benchmark. A balanced synthetic benchmark covers 11 task categories (285 cases each) across 5 mock cluster scenarios. Each case carries ground-truth labels for tool use, routing, HITL confirmation, and source/target state, enabling architecture-level rather than text-quality evaluation.

Architecture-Aware Evaluation System. A dataset-driven evaluation stack measures tool recall, routing match, HITL safety, state transitions, response quality, and optional LLM-as-judge scoring with parallel workers and persistent result artifacts.

Role-Scoped Distillation for Fine-Tuning. A methodology for distilling commercial-model behavior into an open-weight model while preserving the Observer/Operator boundary: training traces are split at handoff points, each sample's declared tool set is restricted to the active agent role, and the fine-tuned model internalizes the separation rather than relying solely on runtime enforcement.

7.1.2 Achievement Against Project Goals

All five project goals are met. Natural-language access to Slurm (Goal 1) is delivered across read, diagnose, account, documentation, and confirmed action workflows. The Observer/Operator control flow (Goal 2) is fully functional and measurable in the benchmark, with the routing dimension explicitly scored. Risk-aware MCP tool integration (Goal 3) is achieved through agent-side allowlists that enforce which tools each role may call, independent of model output. The curated benchmark dataset (Goal 4) contains 3,135 balanced cases with complete ground-truth annotations for all scoring dimensions. Domain-adapted fine-tuning (Goal 5) produces a model that achieves 88.3% mean weighted score on the 615-case held-out test split, closing 29% of the commercial gap while running entirely on local infrastructure.

7.1.3 Key Findings

Architecture-aware metrics are necessary for reliable agentic evaluation: text quality alone does not capture whether destructive operations were correctly gated or whether the right agent executed each step. Deterministic state resets between test cases are essential — without them, state contamination between cases makes pass/fail scores non-reproducible. Trace-based result recording (observed tools, routing decisions, HITL activations, state outcomes) makes failed cases diagnosable rather than reducing them to an opaque score. Finally, role-scoped training data is effective at transferring safety-critical behaviour: the fine-tuned model achieves 88.6% on the safety category and 90.4% overall HITL match, confirming that the confirmation gate internalised during distillation activates reliably at inference time.

7.1.4 Public Artifacts

The source code, fine-tuned QLoRA adapter, and benchmark dataset are publicly released; references are in Section 5.4. The benchmark is an independently reusable artifact and can evaluate any agent or LLM on Slurm operational tasks without running the full system.

7.2 Limitations

The following limitations apply before claims of unrestricted robustness or production-grade deployment are justified.

7.2.1 Evaluation Scope

The benchmark is a manually constructed scenario grid covering 11 categories of common HPC operational patterns across five static cluster snapshots (healthy, failed, pending, mixed, debug_needed), each with the same three users and a fixed job-ID range. Results represent controlled coverage of those patterns; generalisation to arbitrary site policies, non-standard configurations, or phrasings not represented in the grid cannot be assumed without further testing. The static snapshot approach ensures reproducibility but does not capture live cluster volatility, scheduler dynamics, or concurrent user interactions. The routing metric (25% of the overall score) additionally couples evaluation to the dual-agent architecture: the monolithic ablation receives routing = 0 on the 237 handoff cases by construction, so the +6.1 pp routing-excluded gap understates the per-step quality benefit (+7.4 pp tool recall on routing-neutral cases). Table 6.5 makes this metric coupling explicit by reporting both views. Human-in-the-loop user studies would extend coverage to practitioner workflows and natural-language variation beyond the synthetic test set.

7.2.2 Behavioral and Deployment Constraints

Submission, bulk, and multi-step workflows present the hardest evaluation targets, reflecting the genuine complexity of multi-target mutation and dependency-aware scheduling. Training data is generated from a finite mock state space, so the distilled corpus has inherent repetition in some operational patterns. The implementation covers common Slurm workflows but does not encompass every site-specific plugin, wrapper, or local administrative procedure. Production deployment in multi-tenant environments would require fine-grained authentication, tenant isolation, and comprehensive audit logging beyond what the evaluation harness exercises.

7.2.3 Model and Metric Constraints

Behavioural performance depends on model and provider selection; transfer of exact metrics to different model stacks requires re-evaluation. LLM-as-judge scoring introduces model-dependent variance. All reported figures are reproducible from the pinned dataset and model checkpoint identifiers, but results on different hardware or provider versions may differ due to inference non-determinism or model-version drift.

References

[1] Abhishek and B. Thirumala Rao. “Dynamic Assignment of Scientific Computing Resources Using Containers”. In: *International Journal of Computer Sciences and Engineering* 13.2 (2022), pp. 149–156.

[2] ACM HPDC. “LangChain-Parsl: Connect Large Language Model Agents to High Performance Computing”. In: *Proceedings of the 34th International Symposium on High-Performance Parallel and Distributed Computing*. 2025. DOI: 10.1145/3731599.3767349.

[3] AI Plain English. *Built with LangGraph #13: ReAct Agent*. https://ai.plainenglish.io/built-with-langgraph-13-react-agent-1138973e9252. Accessed: 2026-05-10. 2024.

[4] Anthropic. *Model Context Protocol*. https://modelcontextprotocol.io. Accessed: 2024. 2024.

[5] Anthropic. *Model Context Protocol Specification*. https://spec.modelcontextprotocol.io. Accessed: 2024. 2024.

[6] Yuntao Bai et al. “Constitutional AI: Harmlessness from AI feedback”. In: *arXiv preprint arXiv:2212.08073* (2022).

[7] Kinjal Basu, Ibrahim Abdelaziz, Subhajit Farquhar, et al. “API-BLEND: A Comprehensive Corpora for Training and Benchmarking API LLMs”. In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 2024.

[8] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.

[9] Yuheng Cheng et al. “Exploring Large Language Model based Intelligent Agents: Definitions, Methods, and Prospects”. In: *arXiv preprint arXiv:2401.03428* (2024).

[10] Gordon V Cormack, Charles LA Clarke, and Stefan Buettcher. “Reciprocal rank fusion outperforms condorcet and individual rank learning methods”. In: *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*. 2009, pp. 758–759.

[11] Tim Dettmers et al. “QLoRA: Efficient Finetuning of Quantized Language Models”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[12] Ali Doosthosseini et al. “Chat AI: A Seamless Slurm-Native Solution for HPC-Based Services”. In: *arXiv preprint arXiv:2407.00110*. 2024.

[13] Dror G Feitelson, Dan Tsafrir, and David Krakov. “Experience with using the parallel workloads archive”. In: *Journal of Parallel and Distributed Computing* 74.10 (2014), pp. 2967–2982.

[14] Fujitsu Research. *AI Computing Broker: Optimizing AI Computing Efficiency*. https://en-documents.research.global.fujitsu.com/ai-computing-broker/documents/ACB_WhitePaper_en.pdf. 2025.

[15] Grafana Labs. *Grafana Documentation*. https://grafana.com/docs/. Accessed: 2024. 2024.

[16] Ryan Greenblatt et al. *AI Control: Improving Safety Despite Intentional Subversion*. arXiv preprint arXiv:2312.06942. 2024.

[17] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the Knowledge in a Neural Network". In: *arXiv preprint arXiv:1503.02531* (2015).

[18] Xinyi Hou et al. "Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions". In: *arXiv preprint arXiv:2503.23278* (2025).

[19] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2022).

[20] David E. Hudak et al. "Open OnDemand: A web-based client portal for HPC centers". In: *Journal of Open Source Software* 3.25 (2018), p. 622. DOI: 10.21105/joss.00622.

[21] Mohammadali Jamshidi, Maryam Salim, et al. *Model Context Protocol (MCP): Landscape, Security Threats, and Future Research Directions*. arXiv preprint arXiv:2512.06556. 2025.

[22] Jupyter Project. *JupyterHub Documentation*. https://jupyterhub.readthedocs.io/. Accessed: 2024. 2024.

[23] Patrick Lewis et al. "Retrieval-augmented generation for knowledge-intensive NLP tasks". In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 9459–9474.

[24] Xiao Liu et al. "AgentBench: Evaluating LLMs as Agents". In: *Proceedings of the Twelfth International Conference on Learning Representations*. 2024.

[25] Glenn K Lockwood, Drew Hazen, and Nicholas J Wright. "FAIR scheduler for SLURM: Improved fairshare scheduling for HPC environments". In: *2017 IEEE International Conference on Cluster Computing (CLUSTER)*. IEEE. 2017, pp. 550–555.

[26] MDN Web Docs. *Server-Sent Events (SSE)*. https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events. Accessed: 2024. 2024.

[27] Mermaid. *Mermaid: Diagramming and charting tool*. https://mermaid.js.org. Accessed: 2024. 2024.

[28] Seungone Min et al. "GOAT: Goal-Oriented Agent Training for Tool Use". In: *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics*. 2025.

[29] Ollama. *Ollama: Get up and running with large language models locally*. https://ollama.ai. Accessed: 2024. 2024.

[30] OpenAI. "GPT-4 technical report". In: *arXiv preprint arXiv:2303.08774* (2023).

[31] OpenAI. *OpenAI Agents SDK*. https://github.com/openai/openai-agents-python. Accessed: 2024. 2024.

[32] Long Ouyang et al. "Training language models to follow instructions with human feedback". In: *Advances in Neural Information Processing Systems* 35 (2022), pp. 27730–27744.

[33] Jeffrey T. Palmer et al. "Open XDMoD: A tool for the comprehensive management of high-performance computing resources". In: *Computing in Science & Engineering* 17.4 (2015), pp. 52–62. DOI: 10.1109/MCSE.2015.68.

[34] Shishir G Patil et al. "Gorilla: Large Language Model Connected with Massive APIs". In: *arXiv preprint arXiv:2305.15334* (2023).

[35] Prometheus Authors. *Prometheus Monitoring System and Time Series Database*. https://prometheus.io/. Accessed: 2024. 2024.

[36] Yujia Qin et al. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs". In: *arXiv preprint arXiv:2307.16789* (2023).

[37] Sebastian Ramirez. *FastAPI: Modern, fast web framework for building APIs with Python*. https://fastapi.tiangolo.com. Accessed: 2024. 2024.

[38] Nils Reimers and Iryna Gurevych. "Sentence-BERT: Sentence embeddings using Siamese BERT-networks". In: *arXiv preprint arXiv:1908.10084* (2019).

[39] Gregory Sabin and Rebecca Braby. "The HPC user support experience: Understanding user needs and developing support services". In: *Computing in Science & Engineering* 22.1 (2020), pp. 75–82.

[40] SchedMD. *Slurm Commercial Support and Development*. https://www.schedmd.com/. Accessed: 2024. 2024.

[41] SchedMD. *Slurm REST API*. https://slurm.schedmd.com/rest.html. Accessed: 2024. 2024.

[42] SchedMD. *Slurm Workload Manager Documentation*. https://slurm.schedmd.com/documentation.html. Accessed: 2024. 2024.

[43] Timo Schick et al. "Toolformer: Language models can teach themselves to use tools". In: *Advances in Neural Information Processing Systems* 36 (2023).

[44] Noah Shinn et al. "Reflexion: Language agents with verbal reinforcement learning". In: *Advances in Neural Information Processing Systems* 36 (2023).

[45] Ryota Shiraishi et al. *Quantum-HPC Hybrid Execution via the Model Context Protocol*. arXiv preprint arXiv:2604.08318. 2026.

[46] Svelte. *Svelte: Cybernetically enhanced web apps*. https://svelte.dev. Accessed: 2024. 2024.

[47] Hugo Touvron et al. “Llama 2: Open foundation and fine-tuned chat models”. In: *arXiv preprint arXiv:2307.09288* (2023).

[48] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[49] Bailin Wang et al. “RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. 2020, pp. 7567–7578.

[50] Liang Wang et al. “Text embeddings by weakly-supervised contrastive pre-training”. In: *arXiv preprint arXiv:2212.03533* (2022).

[51] Ziyue Wang et al. *Trajectory2Task: Synthesising Verifiable Multi-Step Trajectories for Complex Tool-Use Tasks*. arXiv preprint arXiv:2601.20144. 2026.

[52] Michael Wooldridge. “An introduction to multiagent systems”. In: (2009).

[53] Qingyun Wu et al. “AutoGen: Enabling next-gen LLM applications via multi-agent conversation”. In: *arXiv preprint arXiv:2308.08155* (2023).

[54] Zhiheng Xi et al. “The rise and potential of large language model based agents: A survey”. In: *arXiv preprint arXiv:2309.07864* (2023).

[55] Qiantong Xu et al. *On the Tool Manipulation Capability of Open-source Large Language Models*. arXiv preprint arXiv:2305.16504. 2023.

[56] Shunyu Yao, Noah Shinn, Karthik Narasimhan, et al. “τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains”. In: *arXiv preprint arXiv:2406.12045*. 2024.

[57] Shunyu Yao et al. “ReAct: Synergizing reasoning and acting in language models”. In: *arXiv preprint arXiv:2210.03629* (2022).

[58] Andy B Yoo, Morris A Jette, and Mark Grondona. “SLURM: Simple linux utility for resource management”. In: *Workshop on Job Scheduling Strategies for Parallel Processing*. Springer, 2003, pp. 44–60.

[59] Tao Yu et al. “Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task”. In: *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2018, pp. 3911–3921.

[60] Aohan Zeng et al. “AgentTuning: Enabling Generalized Agent Abilities for LLMs”. In: *arXiv preprint arXiv:2310.12823* (2023).

[61] Lianmin Zheng et al. “Judging LLM-as-a-judge with MT-bench and chatbot arena”. In: *Advances in Neural Information Processing Systems* 36 (2023).

[62] Victor Zhong, Caiming Xiong, and Richard Socher. “Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning”. In: *arXiv preprint arXiv:1709.00103* (2017).